Package 'relieverChangepoint'

August 14, 2026

Title Efficient Changepoint Detection and Model Selection

Version 0.9.0

Maintainer Chengde Qian <qianchd@gmail.com>

Description Detects changepoints in settings where repeatedly fitting models over candidate intervals is computationally expensive. Built-in tools cover mean, regression, kernel, nonparametric, and classifier-based changes, with search methods including segment neighbourhood, optimal partitioning, PELT, wild binary segmentation, seeded binary segmentation, and binary segmentation. Information criteria can select the number of changepoints; hold-out, recycled cross-validation, and outer cross-validation can also select model hyperparameters when the loss family provides comparable candidates. The Reliever methodology is described in Qian, Wang and Zou (2025) <https://www.jmlr.org/papers/v26/24-1108.html>, and the cross-fitting tools are based on Qian, Wang, Wang and Zou (2024) <doi:10.48550/arXiv.2411.07874> and Zou, Wang and Li (2020) <doi:10.1214/19-AOS1814>.

License GPL-3

Encoding UTF-8

SystemRequirements C++17

Roxygen list(markdown = TRUE)

URL https://qianchd.github.io/reliever_changepoint/,
https://github.com/qianchd/reliever_changepoint

BugReports https://github.com/qianchd/reliever_changepoint/issues

LinkingTo Rcpp,
RcppArmadillo

Imports glmnet (>= 5.0.0),
R6,
Rcpp

Suggests nnet,
ranger,
testthat (>= 3.0.0)

Config/testthat/edition 3

Config/roxygen2/version 8.1.0

Config/Needs/website r-lib/pkgdown

Contents

`relieverChangepoint-package`
relieverChangepoint: Efficient Changepoint Detection and Model Selection

---

Description

`relieverChangepoint` provides Reliever-style changepoint detection for settings where fitting the interval model is the expensive step. Built-in functions cover mean, regression, kernel, nonparametric, and classifier-based changes. Information criteria can select the number of changepoints. Hold-out data, recycled cross-validation, and outer cross-validation can also select model hyperparameters when comparable candidates are available.

Primary fitting interfaces

The two primary fitting functions are:

- `reliever()` for built-in losses. It fits mean changes by default; choose another method with `cpd_family`, for example `reliever(X, y, cpd_family = "lasso_crossfit")` for regression changes or `reliever(X, cpd_family = "kde_nll_crossfit")` for distribution changes.
- `reliever_generic(data, reg_fun = ...)` for a user-supplied interval-loss function.

Focused `reliever_*()` functions expose the complete arguments for the same built-in methods.

Fitting and selection

A `reliever()` or `reliever_generic()` call fits candidate changepoint paths with `cpn_crit = "none"` by default. Select a path with one of the following workflows.

For ordinary mean and lasso losses, `cv.reliever()` implements the sample-efficient cross-fitted version of CPSS. CPSS is the sample-splitting selector introduced by Zou, Wang, and Li (2020):

```
mean_cv <- cv.reliever(
  X = x, cpd_family = "mean",
  cpn_max = 5, dm = 30, cov_rate = 0.8
)
plot(mean_cv)
summary(mean_cv)

lasso_cv <- cv.reliever(
  X = X, y = y, cpd_family = "lasso",
  cpn_max = 5, dm = 30, cov_rate = 0.8
```

```
)
plot(lasso_cv)
summary(lasso_cv)
```

Each `cv.reliever()` call automatically performs the final full-data `reliever()` fit after outer-CV selection and returns it in `full_data_fit`. Do not call `reliever()` again merely to refit the selected result.

Use `cv.reliever_generic()` for a compatible custom loss. Fixed-kernel KDE-L2 and NMCD paths support explicit information criteria or independent hold-out selection. KDE bandwidth selection is available through `cpd_family = "kde_nll_crossfit"`.

A cross-fitted fit returns only the interval-adaptive `recv` loss by default. Request homogeneous-hyperparameter paths when they are needed:

```
fit <- reliever(
  X = X, y = y, cpd_family = "lasso_crossfit",
  cpn_crit = "none",
  loss_output_types = c("recv", "crossfit_homo_hyper")
)
plot(fit, run_type = "recv")

recv <- select_by_run(
  result = fit, run_type = "recv", cpn_crit = "loss"
)

recv_homo_hyper <- select_across_runs(
  result = fit, run_type = "crossfit_homo_hyper", cpn_crit = "loss"
)
```

`run_type = "recv"` lets the hyperparameter adapt by interval and selects K and changepoints. `run_type = "crossfit_homo_hyper"` compares cross-fitted paths with one global hyperparameter and jointly selects that setting, K, and changepoints. For lasso this is the normalized `lam_set`. Both selectors reuse `fit`. Add `"incv"` to `loss_output_types` only when the matching interval-adaptive in-sample loss is also required.

Information criteria are explicit alternatives applied with `select_by_run()`; see `reliever()` for the available criteria.

With `cpn_crit = "none"`, `summary(fit)` is usually empty and `plot(fit, run_type = "recv")` shows the interval-adaptive fitted path. A fixed-K hyperparameter curve uses:

```
plot(
  x = fit,
  x_axis = "hyperparameter",
  K = recv$K_hat[[1L]],
  run_type = "crossfit_homo_hyper"
)
```

Display the selected changepoints with `plot_reliever_data(result = fit, data = y, selection = recv)`.

Post-fit selectors

`select_by_run()` Select K separately in each requested loss path.

`select_across_runs()` Select one comparable loss path and K jointly.

`select_holdout()` Select a fitted path using independent evaluation data.

Extending Reliever

`reliever_generic()` accepts a custom interval-loss function (`reg_fun`). Its help page gives the loss-output contract and a minimal implementation. Compatible functions can also be used with `cv.reliever_generic()`.

Search algorithms include segment neighbourhood, optimal partitioning, PELT, globally greedy and recursive wild binary segmentation, seeded binary segmentation, and binary segmentation. `twostep()` and `local_refine()` provide alternative low-model-fit and refinement workflows.

Author(s)

Maintainer: Chengde Qian <qianchd@gmail.com>

Authors:

- Chengde Qian <qianchd@gmail.com>
- Guanghui Wang <ghwang.nk@gmail.com>
- Changliang Zou <chlzou.nk@gmail.com>

References

Qian, C., Wang, G., and Zou, C. (2025). Reliever: Relieving the burden of costly model fits for changepoint detection. *Journal of Machine Learning Research*, 26(203), 1–57.

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint detection in complex models: Cross-fitting is needed. arXiv:2411.07874.

Zou, C., Wang, G., and Li, R. (2020). Consistent selection of the number of change-points via sample-splitting. *The Annals of Statistics*, 48(1), 413–439.

See Also

`reliever()`, `cv.reliever()`, and `reliever_generic()`.

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, mean = 0, sd = 0.5),
  rnorm(n_seg, mean = 4, sd = 0.5),
  rnorm(n_seg, mean = -4, sd = 0.5)
)
fit_cv <- cv.reliever(
  X = x, cpn_max = 5, dm = 30, cov_rate = 0.8,
  nfolds = 3, method = "SN"
)
plot(fit_cv)
selected <- summary(fit_cv)
selected
stopifnot(identical(selected$K_hat, 2L))
```

cp_error *Symmetric changepoint-location error*

Description

Compute the symmetric Hausdorff distance between estimated and true changepoint sets: every estimated point is matched to its nearest true point, every true point is matched to its nearest estimate, and the largest of these distances is returned.

Usage

```
cp_error(estimate, truth)
```

Arguments

`estimate`	Estimated changepoint locations.
`truth`	True changepoint locations.

Value

A non-negative numeric error. The error is zero when both sets are empty and infinite when exactly one set is empty.

See Also

`reliever()`, `reliever_generic()`, `select_by_run()`

Examples

```
cp_error(c(298, 603), c(300, 600))
```

create_relief_itv *Create deterministic relief intervals*

Description

Construct the representative intervals on which Reliever fits models when `cov_rate < 1`. The intervals are arranged in layers of increasing length so that an exact interval requested by a changepoint algorithm can be mapped to a containing representative interval. Endpoints use the half-open convention `(l, r]`, meaning observations `l + 1` through `r`; consequently, `l` may be 0 and `r` may be `n`.

Usage

```
create_relief_itv(n, cov_rate, dm)
```

Arguments

n	Number of observations.
cov_rate	Requested Reliever coverage rate in (0, 1].
dm	Smallest exact-interval length that must be covered. This normally equals the minimum segment length supplied to reliever().

Value

A list describing the layered interval set:

- int_eps: two-column matrix of all (l, r] endpoints, ordered by layer.
- layer_point: cumulative row number at which each layer ends in int_eps.
- int_len: representative-interval length in each layer.
- target_cover_len: exact-interval length from which each layer is designed to provide the requested coverage.
- miss_cover_len: largest exact-interval length that may still fail to contain a representative interval from that layer; the mapping routines use this cutoff to choose the first eligible layer.
- n: number of observations.

See Also

reliever_generic(), reliever_mean(), create_wbs_itv(), create_seed_itv()

Examples

```
int_set <- create_relief_itv(n = 900, cov_rate = 0.8, dm = 30)
head(int_set$int_eps)
int_set$target_cover_len
```

---

create_seed_itv *Create deterministic SeedBS intervals*

Description

Create the multiscale deterministic intervals used by method = "SeedBS". The returned matrix has columns l and r; each row represents observations l + 1 through r.

Usage

```
create_seed_itv(n, dm)
```

Arguments

n	Number of observations.
dm	Target width of the base partition used to build the multiscale seed intervals. The shortest returned intervals span about two base blocks, rather than exactly dm observations.

Value

An integer matrix with columns `l` and `r`.

See Also

`create_wbs_itv()`, `twostep()`, `reliever_generic()`

Examples

```
head(create_seed_itv(n = 900, dm = 30))
```

`create_wbs_itv` *Create random WBS intervals*

Description

Create random intervals for wild binary segmentation style searches. The returned matrix has two columns `l` and `r`; each row represents observations `l + 1` through `r`. Intervals shorter than `dm` are discarded, so the function can return fewer than `M` rows.

Usage

```
create_wbs_itv(n, dm, M, wbs_seed = NULL)
```

Arguments

`n`	Number of observations.
`dm`	Minimum retained interval length.
`M`	Maximum number of retained intervals.
`wbs_seed`	Optional seed used for this interval sample.

Value

An integer matrix with columns `l` and `r`.

See Also

`create_seed_itv()`, `twostep()`, `reliever_generic()`

Examples

```
create_wbs_itv(n = 900, dm = 30, M = 20, wbs_seed = 2026)
```

`cv.reliever` *Cross-validated changepoint detection with built-in loss families*

Description

The sample-efficient cross-fitted version of the CPSS sample-splitting selector of Zou, Wang, and Li (2020). Built-in single-path losses cover changes in the mean, covariance, mean and covariance, linear models, generalized linear models, common exponential-family distributions, fixed-kernel KDE-L2 features, and univariate distributions through NMCD. Lasso additionally supplies a normalized lambda path. Each outer fold reruns the changepoint search on its training rows and scores the fitted segments on held-out rows. Held-out loss selects K and, for lasso, a normalized lambda setting. A final full-data fit supplies the reported changepoints.

Usage

```
cv.reliever(
  X,
  y = NULL,
  cpd_family = "mean",
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  nfolds = 5,
  op_size = 1,
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  ratio = 0.9,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  detail = FALSE,
  echo = FALSE,
  dc_grid_size = NULL,
  ...
)
```

Arguments

`X`	For `"mean"`, `"var"`, `"meanvar"`, `"em"`, and `"nmcd"`, a numeric vector or matrix with observations in rows; NMCD requires univariate input. For `"kde_l2"`, a precomputed kernel-feature matrix, or raw observations together with a fixed `kernel` and `bandwidth`. For `"lm"`, `"glm"`, and `"lasso"`, a predictor matrix when `y` is supplied, or a matrix whose leading column or columns contain the response when `y = NULL`. Lasso requires at least two predictor columns.
`y`	Optional response for `"lm"`, `"glm"`, and `"lasso"`. A binomial GLM may use a two-column matrix of successes and failures. If omitted, the first column of `X` is used as the response by default. It is ignored with a warning for non-regression families.

cpd_family	Built-in outer-CV loss family: "mean", "var", "meanvar", "lm", "glm", "em", "lasso", "kde_l2", or "nmcd".
cpn_max, cov_rate, method	Common path controls; see reliever().
dm	Minimum segment length for the full-data fit. It is rescaled proportionally inside each training fold.
nfolds	Number of global outer folds. The sample size need not be divisible by nfolds; remainder observations are distributed across folds.
op_size	Number of consecutive observations assigned to one outer fold before assignment cycles to the next fold.
pen_val, prune_value, M, wbs_seed, wbs_stop_crit	Common penalty and search controls; see reliever().
ratio	Computational update threshold in (0, 1] used only for ungridded mean loss. It affects speed, not the criterion. An explicitly supplied value is ignored with a warning for other cases.
cache_backend, owner_key, echo	Common cache and output controls; see reliever().
detail	Detail level. TRUE or "cache" retains fold assignments, losses, and final-fit cache information. Use FALSE or "none" for a compact result.
dc_grid_size	Preferred regular-grid spacing for outer CV. The full-data grid uses this spacing and ends at n; each training-fold spacing is scaled in proportion to its sample size. The default NULL searches every boundary. See reliever() for the candidate-grid motivation and reference.
...	Model-specific arguments passed to the selected reg_fun; see that function's help page.

Details

As part of the same call, cv.reliever() automatically runs the final reliever() fit on all observations. The returned full_data_fit contains this fit, and summary() reports its outer-CV-selected K and changepoints. Do not call reliever() again after cv.reliever() merely to refit the selected result.

Training rows retain their order, and fold-specific changepoints are mapped to the original time axis before held-out observations are scored.

Candidates are matched by K, wbs_stop_crit, or pen_val, as appropriate. Only candidates available in every training fold and the final full-data path are compared. WBS-family and PELT/OP comparisons also contain an implicit Inf-valued no-change baseline, which may be selected.

The default mean family uses mean-square loss. The "var" and "meanvar" families use multivariate Gaussian negative twice log-likelihood. The "lm" family uses squared residuals, "glm" uses family deviance residuals, and "em" uses negative twice log-likelihood for one selected distribution. These are single-path families: outer CV selects K, not a model or distribution family. Fixed-kernel "kde_l2" evaluates held-out kernel feature rows against segment centers, while "nmcd" evaluates held-out empirical-CDF loss using fold-specific training references. For KDE-L2, the kernel and bandwidth are fixed before outer CV; this interface selects K rather than tuning the bandwidth.

The lasso family evaluates glmnet prediction loss over a normalized lam_set; a fit with m rows passes lam_set/ sqrt(m) to glmnet. When lam_set = NULL, one full-data path, with length requested through nlambda, is generated and reused in all folds. An explicit lam_set takes precedence over nlambda. Lasso fits use no intercept or automatic predictor standardization.

For interval-level cross-fitting, use reliever() with cpd_family = "lasso_crossfit", followed by select_by_run(..., run_type = "recv", cpn_crit = "loss").

Value

An object of class `cv_reliever_result` with:

`summary` The selected K, changepoints, model setting when applicable, and mean CV loss with its standard error. The `summary()` method returns this row.

`cv_loss` All compared candidates with their mean held-out loss and standard error. Use `plot()` to visualize this table.

`full_data_fit` The automatically fitted final full-data `reliever()` result and its candidate path. No additional external `reliever()` call is required.

`settings` Fold and full-data search settings.

`diagnostics` Fold assignments and losses when requested.

References

Qian, C., Wang, G., and Zou, C. (2025). Reliever: Relieving the burden of costly model fits for changepoint detection. *Journal of Machine Learning Research*, 26(203), 1–57.

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint detection in complex models: Cross-fitting is needed. arXiv:2411.07874.

Zou, C., Wang, G., and Li, R. (2020). Consistent selection of the number of change-points via sample-splitting. *The Annals of Statistics*, 48(1), 413–439.

See Also

`reliever()`, `cv.reliever_generic()`, and `select_by_run()`.

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, mean = 0, sd = 0.5),
  rnorm(n_seg, mean = 4, sd = 0.5),
  rnorm(n_seg, mean = -4, sd = 0.5)
)
fit <- cv.reliever(
  X = x,
  cpn_max = 5, dm = 30, cov_rate = 0.7,
  method = "SN", nfolds = 5,
  dc_grid_size = 10
)
selected <- summary(fit)
selected
stopifnot(identical(selected$K_hat, 2L))
# fit$full_data_fit is already the final reliever fit on all observations.
plot(fit)

# CPSS outer CV also selects K for NMCD and fixed-kernel KDE-L2.
set.seed(2026)
n_nmcd <- 90
x_nmcd <- c(
  rnorm(n_nmcd, -4, 0.35),
  rnorm(n_nmcd, 4, 0.35),
  rnorm(n_nmcd, 0, 0.35)
```

```
)
fit_nmcd <- cv.reliever(
  X = x_nmcd, cpd_family = "nmcd",
  cpn_max = 4, dm = 18, cov_rate = 0.7,
  method = "SN", nfolds = 5, dc_grid_size = 10
)
stopifnot(identical(summary(fit_nmcd)$K_hat, 2L))

set.seed(2026)
n_kde <- 50
x_kde <- c(
  rnorm(n_kde, -3, 0.45),
  rnorm(n_kde, 3, 0.45),
  rnorm(n_kde, 0, 0.45)
)
fit_kde_l2 <- cv.reliever(
  X = x_kde, cpd_family = "kde_l2",
  kernel = "gaussian", bandwidth = 0.8,
  cpn_max = 4, dm = 10, cov_rate = 0.6,
  method = "SN", nfolds = 5, dc_grid_size = 5
)
stopifnot(identical(summary(fit_kde_l2)$K_hat, 2L))


# This high-dimensional lasso example takes more than five seconds.
# Lasso outer CV jointly selects one automatic-path setting and K.
p <- 20
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
set.seed(2026)
reg_data <- dgp_linear_regression(
  n = 450, p = p, tau = c(150, 300),
  b0 = b0, delta = delta, sig = 1
)$data
fit_lasso <- cv.reliever(
  X = reg_data[, -1, drop = FALSE], y = reg_data[, 1],
  cpd_family = "lasso",
  cpn_max = 5, dm = 15, cov_rate = 0.6, method = "SN",
  nfolds = 3, nlambda = 20
)
plot(fit_lasso)
selected_lasso <- summary(fit_lasso)
selected_lasso
stopifnot(identical(selected_lasso$K_hat, 2L))


# The other single-path parametric families use the same outer-CV selector.
set.seed(2027)
n <- 100
z <- c(rnorm(50, sd = 0.5), rnorm(50, sd = 2))
fit_var <- cv.reliever(
  X = z, cpd_family = "var", cpn_max = 2, dm = 8,
  cov_rate = 0.6, method = "BS", nfolds = 2
)
plot(fit_var)
fit_meanvar <- cv.reliever(
  X = z, cpd_family = "meanvar", cpn_max = 2, dm = 8,
```

```
  cov_rate = 0.6, method = "BS", nfolds = 2
)
plot(fit_meanvar)

predictor <- rnorm(n)
coefficient <- rep(c(2, -2), each = 50)
response <- coefficient * predictor + rnorm(n, sd = 0.25)
fit_lm <- cv.reliever(
  X = matrix(predictor, ncol = 1), y = response, cpd_family = "lm",
  cpn_max = 2, dm = 8, cov_rate = 0.6, method = "BS", nfolds = 2
)
plot(fit_lm)

trials <- rep(20L, n)
success <- rbinom(n, trials, plogis(2.5 * coefficient * predictor))
fit_glm <- cv.reliever(
  X = matrix(predictor, ncol = 1),
  y = cbind(success, trials - success), cpd_family = "glm",
  family = binomial(), cpn_max = 2, dm = 8,
  cov_rate = 0.6, method = "BS", nfolds = 2
)
plot(fit_glm)

exponential <- c(rexp(50, 1), rexp(50, 5))
fit_em <- cv.reliever(
  X = exponential, cpd_family = "em", family = "exp",
  cpn_max = 2, dm = 8, cov_rate = 0.6, method = "BS", nfolds = 2
)
plot(fit_em)

selected_k <- vapply(
  list(fit_var, fit_meanvar, fit_lm, fit_glm, fit_em),
  function(object) object$summary$K_hat,
  integer(1)
)
selected_k
stopifnot(identical(selected_k, rep(1L, 5L)))
```

`cv.reliever_generic` *Outer cross-validation for generic Reliever models*

Description

CPSS-style outer cross-validation for a user-supplied interval-loss function. Each outer training fold runs `reliever_generic()`; held-out loss selects a model setting and candidate path, and a final full-data fit supplies the reported K and changepoints. This final `reliever_generic()` fit is performed automatically and returned in `full_data_fit`; users do not need to call `reliever_generic()` again after model selection. By default all declared loss outputs are compared; `run_cpd_ids` selects a subset.

Usage

```
cv.reliever_generic(
  data,
```

```
  reg_fun,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  nfolds = 5,
  op_size = 1,
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  run_cpd_ids = NULL,
  data_folding_fun = NULL,
  data_stack_fun = NULL,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  detail = FALSE,
  echo = FALSE,
  dc_grid_size = NULL,
  ...
)
```

Arguments

`data`, `reg_fun`	Custom data object and interval-loss function; see `reliever_generic()`.
`cpn_max`, `dm`, `cov_rate`, `method`	Common path controls; see `cv.reliever()`.
`nfolds`, `op_size`	Common outer-CV controls; see `cv.reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `cv.reliever()`.
`run_cpd_ids`	Optional declared loss-output identifiers to compare. The default compares every output supplied by `reg_fun`.
`data_folding_fun`	Advanced function called as `data_folding_fun(data, train_id, eval_id)`. It must return a list containing `train_data` and `eval_data`, with rows kept in the respective `train_id` and `eval_id` orders. The default subsets row-oriented vectors, matrices, and data frames.
`data_stack_fun`	Advanced function that combines a fold's training and held-out data into the object expected by `reg_fun`. The default row-binds vectors, matrices, and data frames.
`cache_backend`, `owner_key`, `detail`, `echo`	Common output and cache controls; see `cv.reliever()`.
`dc_grid_size`	Candidate-grid control; see `cv.reliever()`.
`...`	Additional arguments passed to `reg_fun`. Outer CV accepts `dc_grid_size`, but not `dc_grid`, `cache_profile`, or `cpn_crit`.

Details

Use `data_folding_fun` and `data_stack_fun` for custom data representations. Compare only outputs with the same held-out scoring rule, units, and normalization; select them with `run_cpd_ids`.

Built-in lasso-crossfit and KDE-NLL-crossfit losses provide model-specific external scorers. Other crossfit templates require a model-specific external scorer before they can be used here.

Value

A `cv_reliever_result` with:

`summary` The selected K, changepoints, loss output when applicable, and mean outer-CV loss with its standard error. `summary()` returns this row.

`cv_loss` Every compared loss output and candidate path, with mean held-out loss and standard error.

`full_data_fit` The automatically fitted final full-data `reliever_generic()` result and its complete candidate paths. No additional external refit is required.

`settings` The outer-fold and full-data search settings.

`diagnostics` Fold assignments and losses when requested.

References

Qian, C., Wang, G., and Zou, C. (2025). Reliever: Relieving the burden of costly model fits for changepoint detection. *Journal of Machine Learning Research*, 26(203), 1–57.

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint detection in complex models: Cross-fitting is needed. arXiv:2411.07874.

Zou, C., Wang, G., and Li, R. (2020). Consistent selection of the number of change-points via sample-splitting. *The Annals of Statistics*, 48(1), 413–439.

See Also

`cv.reliever()`, `reliever_generic()`, and `reg_fun_crossfit_template()`.

Examples

```
set.seed(2026)
x <- c(rnorm(40), rnorm(40, 3), rnorm(40, -3))
absolute_location_loss <- function(data, l, r, l_end = l, r_end = r,
                                   save_model = FALSE,
                                   is_virtual_run = FALSE) {
  if (is_virtual_run) {
    return(1L)
  }
  center <- stats::median(data[l:r])
  list(
    loss = matrix(abs(data[l_end:r_end] - center), ncol = 1L),
    model = if (save_model) list(center = center) else NULL
  )
}

fit <- cv.reliever_generic(
  data = x,
  reg_fun = absolute_location_loss,
  cpn_max = 3, dm = 10, cov_rate = 0.6,
  method = "SN", nfolds = 3,
  dc_grid_size = 5
)
selected <- summary(fit)
```

```
selected
stopifnot(identical(selected$K_hat, 2L))
```

`dgp_linear_regression` *Generate a linear regression sequence with coefficient changepoints*

Description

Generate observations from a piecewise linear regression model. The response is stored in the first column and predictors are stored in the remaining columns, matching the response-first input convention used by `reg_fun_lasso_solpath()`. For the primary built-in interface, pass the predictor columns and response separately to `reliever(X, y, cpd_family = "lasso")`.

Usage

```
dgp_linear_regression(n, p, tau, b0, delta, sig = 1, rho = 0, rho_ar1 = 0)
```

Arguments

`n`	Number of observations.
`p`	Number of predictors.
`tau`	Increasing integer changepoints. At `t`, the coefficient changes after observation `t`.
`b0`	Coefficient vector in the first segment, of length `p`.
`delta`	A `p` by `length(tau)` matrix. Column `j` is added to the current coefficient vector after changepoint `tau[j]`, so coefficient changes accumulate across segments.
`sig`	Noise standard deviation.
`rho`	Correlation between adjacent predictor coordinates. The within-observation predictor covariance has entries $rho^{\|j-k\|}$. Use 0 for independent predictor coordinates.
`rho_ar1`	AR(1) correlation over time, applied separately to each predictor coordinate and to the response noise. Predictors retain unit marginal variance before spatial correlation, and response innovations have unit variance before either `sig` scaling or segment-specific scaling. `rho_ar1 = 0` gives temporally independent observations.

Value

A list containing `data`, the response followed by predictors; `mu`, the noise-free conditional response mean; and `ep`, the response noise added to `mu`.

See Also

`dgp_linear_regression_equal_response()`, `reliever()`, `cv.reliever()`

Examples

```
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
generated <- dgp_linear_regression(n, p, tau, b0, delta, sig = 0.5)
dim(generated$data)

set.seed(2026)
generated_ar <- dgp_linear_regression(
  n, p, tau, b0, delta, sig = 0.5, rho_ar1 = 0.3
)
dim(generated_ar$data)
```

---

dgp_linear_regression_equal_response

Generate a linear regression sequence with equalized response variance

---

Description

Generate a piecewise linear regression sequence while choosing a separate noise variance in each segment so that the marginal response variance is constant across segments. This avoids making coefficient changes detectable merely through a change in response variance.

Usage

```
dgp_linear_regression_equal_response(
  n,
  p,
  tau,
  b0,
  delta,
  rho = 0,
  snr_y = 1,
  rho_ar1 = 0
)
```

Arguments

n	Number of observations.
p	Number of predictors.
tau	Increasing integer changepoints. At t, the coefficient changes after observation t.
b0	Coefficient vector in the first segment, of length p.
delta	A p by length(tau) matrix. Column j is added to the current coefficient vector after changepoint tau[j], so coefficient changes accumulate across segments.

rho	Correlation between adjacent predictor coordinates. The within-observation predictor covariance has entries $rho^{\|j-k\|}$. Use 0 for independent predictor coordinates.
snr_y	Positive response-level signal-to-noise control. Larger values reduce the common target response variance toward the largest segment-wise signal variance and therefore reduce added noise.
rho_ar1	AR(1) correlation over time, applied separately to each predictor coordinate and to the response noise. Predictors retain unit marginal variance before spatial correlation, and response innovations have unit variance before either `sig` scaling or segment-specific scaling. `rho_ar1 = 0` gives temporally independent observations.

Value

A list containing `data`, the response followed by predictors; `mu`, the noise-free conditional response mean; and `ep`, the segment-scaled response noise.

See Also

`dgp_linear_regression()`, `reliever()`, `cv.reliever()`, `reliever_lasso()`

Examples

```
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
generated <- dgp_linear_regression_equal_response(
  n, p, tau, b0, delta, snr_y = 2
)
dim(generated$data)
```

---

evaluate_reliever_segments

Evaluate candidate segmentations on external data

Description

Compute segment loss on external evaluation data after candidate segmentations have been fitted by `reliever()`. This is a lower-level helper used by `select_holdout()`; most users doing hold-out model selection can call `select_holdout()` directly.

Usage

```
evaluate_reliever_segments(
  result,
  data,
  eval_data,
  eval_index = NULL,
  y = NULL,
```

```
  eval_y = NULL,
  save_model = FALSE,
  run_ids = NULL,
  reg_fun = NULL,
  data_stack_fun = NULL,
  ...,
  run_type = NULL
)
```

Arguments

`result`	A fitted `reliever_result` object.
`data`	Training observations in the representation expected by the fitted loss. For a built-in lasso fit, supply either the predictor matrix together with `y`, or a response-first matrix and omit `y`. A built-in KDE fit made from raw observations accepts those observations again and reconstructs the required training pairwise matrix. For a precomputed KDE-NLL fit, supply the training squared-distance matrix; for a precomputed KDE-L2 fit, supply the training feature or Gram matrix.
`eval_data`	Independent observations on which segment losses are evaluated. For a built-in lasso fit, supply either the evaluation predictor matrix together with `eval_y`, or a response-first matrix and omit `eval_y`. A raw-data KDE fit accepts raw evaluation observations. A precomputed KDE-NLL fit instead requires an evaluation-by-training squared-distance matrix, and a precomputed KDE-L2 fit requires an evaluation-by-feature or evaluation-by-training matrix. It cannot be `NULL`.
`eval_index`	Integer time index for each row of `eval_data`. This tells the function which estimated segment contains that row. For one independent evaluation observation at every original time point, the default `NULL` uses `seq_len(n)`, where n is the number of training observations. If evaluation observations are available only at times 10, 20, and 30, use `c(10, 20, 30)`. Repeated time indices are allowed.
`y`, `eval_y`	Optional training and evaluation responses for a built-in lasso or lasso-crossfit result. Supply both when `data` and `eval_data` contain predictors only. Omit both when their first columns already contain the responses. Do not supply these arguments for custom or non-response loss families.
`save_model`	Logical scalar. If `TRUE`, return the segment model objects produced by `reg_fun`.
`run_ids`	Optional identifiers from `result$run_meta`. The default evaluates every fitted run. Use this to evaluate only selected model settings after fitting; `run_cpd_ids` is instead the fitting-time argument used by `reliever_generic()` to choose which loss outputs become runs. Supply only one of `run_ids` and `run_type`.
`reg_fun`	Optional interval-loss function used to refit and score each segment. Results returned by `reliever()`, its model-specific wrappers, `reliever_generic()`, and `twostep()` retain the original function automatically, so ordinary calls should leave this as `NULL`. Supply it only for a manually constructed result or an intentional compatible override.
`data_stack_fun`	Optional function that stacks `data` and `eval_data` before scoring. The default row-binds vectors, matrices, and data frames. Custom representations may supply another function; its result must put all training rows before all evaluation rows.
`...`	Named loss-function arguments for a manually constructed result, additional evaluation-only arguments, or explicit overrides of retained arguments. An override must preserve the selected loss-output identifiers and hyperparameter path.

`run_type` Optional character values matched against `result$run_meta$row_type`, such as `"recv"` or `"crossfit_homo_hyper"`. Supply only one of `run_type` and `run_ids`.

Details

For each candidate changepoint set, the sample is split into final segments. Within each segment, the model is fit on the corresponding rows of `data`, and loss is evaluated on rows of `eval_data` whose `eval_index` values fall in that segment.

For built-in lasso-crossfit and KDE-NLL-crossfit results, inner tuning uses only each final training segment. The selected model is refitted on that segment and scored on `eval_data`; paired cross-fitted and in-sample outputs therefore share the corresponding external score. Other crossfit templates require a model-specific external scorer.

If a WBS-family fit supplied `wbs_stop_crit`, evaluation is restricted to the segmentations mapped from those thresholds and the no-change baseline. Omit `wbs_stop_crit` when the full K path is needed.

Value

A list with:

`candidates` The original candidate segmentations and their total external `eval_loss`.

`models` Optional fitted models, named `"left:right"` by their training segment.

See Also

`select_holdout()`, `reliever()`, `reliever_generic()`, `reg_fun_lasso_solpath()`

Examples

```
# Constructing and evaluating the KDE-L2 path takes more than five seconds.
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
holdout <- x + matrix(rnorm(length(x), sd = 0.2), nrow = nrow(x))
bandwidth <- sqrt(10)
res <- reliever(
  X = x, cpd_family = "kde_l2",
  kernel = "gaussian", bandwidth = bandwidth,
  cpn_max = 7, dm = 30, cov_rate = 0.7, method = "SN"
)
eval_res <- evaluate_reliever_segments(
  result = res, data = x, eval_data = holdout
)
eval_res$candidates
```

`local_refine` *Locally refine a changepoint set*

Description

Refine each supplied changepoint by minimizing left/right split loss in a local window bounded by its neighboring initial changepoints. Refinement is performed separately for every loss output returned by `reg_fun`.

Usage

```
local_refine(data, tau_init, reg_fun, dm = 50, r = 0.1, echo = FALSE, ...)
```

Arguments

`data`	Data object accepted by `reg_fun`.
`tau_init`	Integer initial changepoint locations. The values are sorted and duplicates are removed before refinement; each value represents one changepoint to refine.
`reg_fun`	Custom interval-loss function following the generic fitting contract.
`dm`	Minimum segment length used in local split evaluation.
`r`	Neighborhood shrinkage parameter in `[0, 1]`. Larger values search closer to each initial changepoint. Zero uses the full neighboring interval; one returns the initial locations unchanged.
`echo`	Print timing messages.
`...`	Additional arguments passed to `reg_fun`.

Value

A list with `tau_est`, `run_meta`, and `total_time`. `tau_est` has one row per loss output returned by `reg_fun` and one column per initial changepoint. `run_meta` describes those rows using the same metadata contract as `reliever_generic()`. For a single-loss model, read the refined changepoints from `tau_est[1, ]`.

See Also

`twostep()`, `reliever_generic()`, `reg_fun_lasso_solpath()`, `reg_fun_mean()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
# Refine two initial changepoints.
refined <- local_refine(x, tau_init = c(295, 605), dm = 30,
                        reg_fun = reg_fun_mean)
refined$tau_est
```

plot.cv_reliever_result *Plot held-out losses from cross-validated Reliever*

Description

Plot mean held-out loss against K for every fitted model setting. The point with the smallest held-out loss among the plotted rows is circled. With the default, unfiltered K path this is the outer-CV selection. For solution-path models, `x_axis = "hyperparameter"` shows held-out loss across numeric or categorical tuning values; categorical values follow their declared run order. Supplying `K` holds the changepoint number fixed while checking whether a lambda or bandwidth grid spans its minimum. For a numeric grid, an endpoint minimum triggers a suggestion to consider a wider grid.

Usage

```
## S3 method for class 'cv_reliever_result'
plot(
  x,
  x_axis = c("K", "hyperparameter", "search_value"),
  K = NULL,
  run_ids = NULL,
  show_se = TRUE,
  col = NULL,
  lty = 1,
  pch = 19,
  xlab = NULL,
  ylab = NULL,
  main = NULL,
  ...,
  run_type = NULL
)
```

Arguments

`x`	A result returned by `cv.reliever()` or `cv.reliever_generic()`.
`x_axis`	Horizontal-axis choice: • `"K"` for changepoint number; • `"hyperparameter"` for values from `x$cv_loss`; or • `"search_value"` for WBS, PELT, or OP controls.
`K`	Optional non-negative changepoint number used only for a fixed-K hyperparameter plot. With `K = NULL`, the best K is selected within each hyperparameter before plotting.
`run_ids`	Optional positive integer run identifiers to display. For a crossfit loss, a hyperparameter plot defaults to its `"crossfit_homo_hyper"` runs. Supply only one of `run_ids` and `run_type`.
`show_se`	Draw one-standard-error bars when available.
`col`, `lty`, `pch`	Graphical parameters for curves and points.
`xlab`, `ylab`, `main`	Optional axis labels and title.

...	Additional arguments passed to graphics::plot.default().
run_type	Optional character values matched against the row_type metadata of the full-data fit. Supply only one of run_type and run_ids.

Value

The rows of x$cv_loss used for plotting, invisibly, with a logical selected column identifying the minimum among those rows. When the full-data run metadata supplies hyper_name, tuning-path plots use it instead of the generic word "hyperparameter".

See Also

summary.cv_reliever_result(), plot.reliever_result(), cv.reliever()

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- cv.reliever(
  X = x, cpn_max = 20, dm = 30, cov_rate = 0.8,
  method = "WBS", M = 200,
  wbs_stop_crit = c(5, 10, 15, 20), nfolds = 3
)
stopifnot(identical(summary(fit)$K_hat, 2L))
plot(fit, x_axis = "search_value")
```

---

plot.reliever_result *Plot Reliever model-selection profiles*

Description

Plot candidate scores against the number of changepoints, inspect a solution path across numeric or categorical hyperparameter values, or plot a WBS stopping criterion or PELT/OP penalty on its original scale. The default uses the selection criterion stored in the fit. Set cpn_crit = "loss" to show and minimize the unpenalized fitted loss instead.

Usage

```
## S3 method for class 'reliever_result'
plot(
  x,
  x_axis = c("K", "hyperparameter", "search_value"),
  K = NULL,
  run_ids = NULL,
  cpn_crit = NULL,
  col = NULL,
  lty = 1,
```

```
  pch = 19,
  xlab = NULL,
  ylab = NULL,
  main = NULL,
  ...,
  run_type = NULL
)
```

Arguments

x	A result returned by reliever() or a related wrapper.
x_axis	Horizontal-axis choice: • "K" for changepoint number; • "hyperparameter" for values from x$run_meta; or • "search_value" for WBS, PELT, or OP controls.
K	Optional non-negative changepoint number. It is used only for a fixed-K hyperparameter plot.
run_ids	Optional positive integer run identifiers. By default, loss outputs marked as preferred are used when available; for cross-fitted families this is the recv run. Otherwise all runs are shown. Supply only one of run_ids and run_type.
cpn_crit	Vertical-axis criterion. The default NULL uses the criterion stored in the fit. Use "loss" for raw fitted loss; the post-fit selector documentation lists the other criteria.
col, lty, pch	Graphical parameters for curves and points.
xlab, ylab, main	Optional axis labels and title.
...	Additional arguments passed to graphics::plot.default().
run_type	Optional stored row types, such as "recv" or "crossfit_homo_hyper". Supply only one of run_type and run_ids.

Details

With x_axis = "K", one curve is drawn for every requested run. A selected K is circled only when the fit stores a selection criterion or cpn_crit is supplied. With x_axis = "hyperparameter", K fixes the changepoint number and produces a fixed-K loss or criterion curve. If K = NULL, a stored or supplied criterion is required and first selects K separately within each run. Hyperparameter plots require one non-missing hyper_value per run in x$run_meta. Numeric values use their numeric scale; categorical values follow their declared run order. If the corresponding metadata also contains one common hyper_name, that name is used for the horizontal-axis label and plot title.

Raw fitted losses from an ordinary solution path are not generally comparable across hyperparameters. Such a profile is therefore shown without circling a preferred hyperparameter or interpreting an endpoint minimum. For a ReCV result, omitting both run_ids and run_type uses the default recv run. Select fixed-hyperparameter cross-fitted runs explicitly with run_type = "crossfit_homo_hyper". When a selection criterion is stored or supplied, those comparable losses have their minimum circled, and a fixed-K minimum at either grid endpoint triggers a warning because the candidate range may be too narrow. Outer-CV profiles have the same grid-check behavior; see plot.cv_reliever_result().

With x_axis = "search_value", the horizontal axis is the supplied wbs_stop_crit or pen_val. This is useful when several search values produce the same K. If the fit has cpn_crit = "none", the loss profile is shown without circling a supposedly selected K. Supply an explicit cpn_crit, including "loss", when a selected point is wanted.

Two-step searches do not compute a comparable whole-segmentation loss, so their candidate table cannot be drawn as a loss or IC profile. Inspect `x$cpd_path$candidates` instead.

Value

The data used for plotting, invisibly. It includes a logical `selected` column identifying the circled points. This column is all `FALSE` when the fit has no K-selection rule and none is supplied, or when an ordinary fitted-loss solution path is plotted across hyperparameters that are not directly comparable.

See Also

`summary.reliever_result()`, `select_by_run()`, `plot.cv_reliever_result()`

Examples

```
set.seed(2026)
x <- rbind(
  matrix(rnorm(300 * 3), 300, 3),
  matrix(rnorm(300 * 3, mean = 3), 300, 3),
  matrix(rnorm(300 * 3, mean = -3), 300, 3)
)
fit <- reliever(X = x, cpn_max = 5, dm = 30, cov_rate = 0.8)
plot(x = fit)
```

---

plot_reliever_data *Plot data with selected changepoints*

Description

Draw one or more observed series against their row index and add vertical lines at selected changepoints. Compact Reliever results do not retain the fitting input, so the data are supplied explicitly. A non-native fit made with `detail = TRUE` or `"cache"` may retain it inside its reusable cache profile.

Usage

```
plot_reliever_data(
  result,
  data,
  selection = NULL,
  columns = 1L,
  type = "l",
  col = NULL,
  lty = 1,
  cpd_col = 2,
  cpd_lty = 2,
  cpd_lwd = 1,
  legend = TRUE,
  xlab = "Observation",
  ylab = NULL,
  main = "Data and selected changepoints",
  ...
)
```

Arguments

result	A reliever_result or cv_reliever_result.
data	The numeric vector, matrix, or data frame used for fitting. Its number of rows must match result$settings$n.
selection	Which changepoint selection to plot. The default NULL uses summary(result) when it has exactly one row. A positive integer chooses one summary row. Alternatively, supply a one-row data frame containing a cpd_hat list-column.
columns	Numeric or character columns of data to draw. The default draws the first column.
type, col, lty	Graphical parameters for the observed series.
cpd_col, cpd_lty, cpd_lwd	Graphical parameters for the changepoint lines.
legend	Add a legend when more than one series is drawn.
xlab, ylab, main	Optional axis labels and title.
...	Additional arguments passed to graphics::matplot().

Details

By default, the function uses the only row of summary(result). If the summary has several rows, such as one per solution-path hyperparameter, supply either a row number or a one-row selection table. A row returned by select_by_run(), select_across_runs(), or select_holdout() can therefore be plotted directly.

Changepoint locations identify the final observation in the preceding segment, so their vertical lines are drawn halfway between that observation and the next one.

Value

The plotted data, invisibly, with the selected changepoints in the "cpd_hat" attribute.

See Also

plot.reliever_result(), select_by_run()

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, 0, 0.5),
  rnorm(n_seg, 4, 0.5),
  rnorm(n_seg, -4, 0.5)
)
fit <- reliever(X = x, cpn_max = 7, dm = 30, cov_rate = 0.8)
selected <- select_by_run(result = fit, cpn_crit = "rss_sic")
stopifnot(identical(selected$K_hat, 2L))
plot_reliever_data(result = fit, data = x, selection = selected)
```

print.cv_reliever_result

Print a cross-validated Reliever result

Description

Display the final full-data changepoint estimate selected by outer cross-validation. The printed columns are the same compact columns returned by `summary(x)`; fold-level candidate losses remain in `x$cv_loss`. The built-in loss family, search method, and number of folds are printed above the result when available.

Usage

```
## S3 method for class 'cv_reliever_result'
print(x, max_rows = 10L, ...)
```

Arguments

`x`	A result returned by `cv.reliever()` or `cv.reliever_generic()`.
`max_rows`	Maximum number of result rows printed. Cross-validated results normally contain one row; the default is 10.
`...`	Additional arguments passed to `print.data.frame()`.

Value

`x`, invisibly. Changepoint locations are printed in full; `x$summary` retains them as a list-column for programmatic use.

print.reliever_model_selection

Print a Reliever model-selection result

Description

Display the statistical result without the internal path identifiers. When the fitted object contains several statistically distinct `row_type` values, that column remains visible so a cross-fitted selection is identifiable as `"recv"`, `"crossfit_homo_hyper"`, or another declared type. The returned object still contains `run_id` and `candidate_id`; set `details = TRUE` to print them when tracing the selection back to `result$run_meta` and `result$cpd_path$candidates`.

Usage

```
## S3 method for class 'reliever_model_selection'
print(x, details = FALSE, max_rows = 10L, ...)
```

Arguments

x	A model-selection result returned by a post-fit selector.
details	Show the internal run and candidate identifiers.
max_rows	Maximum number of selected model settings printed. The default is 10; use Inf to print every row.
...	Additional arguments passed to print.data.frame().

Value

x, invisibly. Changepoint locations are printed in full; the returned object retains them as a list-column for programmatic use.

---

print.reliever_result *Print a Reliever result*

Description

Display any explicitly configured selection in compact form. If the fit was created with cpn_crit = "none", supplied WBS stopping thresholds or PELT/OP penalties are shown directly. If neither a selection rule nor such search values were supplied, or no loss output was marked as eligible for the explicit fit-time criterion, the method points to the complete candidate path in x$cpd_path. When several hyperparameter values are printed, K has been selected separately within each value; the hyperparameters themselves have not been compared.

Usage

```
## S3 method for class 'reliever_result'
print(x, max_rows = 10L, ...)
```

Arguments

x	A result returned by reliever() or a related wrapper.
max_rows	Maximum number of result rows printed. The default is 10; use Inf to print every model setting.
...	Additional arguments passed to print.data.frame().

Value

x, invisibly. Changepoint locations are printed in full; x$summary retains them as a list-column for programmatic use.

See Also

summary.reliever_result(), plot.reliever_result(), reliever(), reliever_generic()

print.reliever_summary

Print a compact Reliever summary

Description

Print every changepoint location without changing the data-frame interface returned by `summary()` and stored in the corresponding result object's `summary` component. When the rows select K separately for several hyperparameter values, the print method also explains that no hyperparameter has been selected across those rows. An empty summary explains that fitting and model selection are separate steps and points to the recommended workflows in `reliever()`.

Usage

```
## S3 method for class 'reliever_summary'
print(x, max_rows = 10L, ...)
```

Arguments

`x`	A compact summary from a Reliever or outer-CV result.
`max_rows`	Maximum number of rows printed. Use `Inf` to print all rows.
`...`	Additional arguments passed to `print.data.frame()`.

Value

`x`, invisibly.

reg_fun_clf_crossfit_template

Cross-fitted probability-classifier interval-loss template

Description

Specialization of `reg_fun_crossfit_template()` for distribution-change detection by binary classification. The template labels observations inside the fitted interval as class 1 and observations outside it as class 0. User code supplies only `prob_fun`, which fits the classifier and predicts a class-1 probability for each requested evaluation row.

Usage

```
reg_fun_clf_crossfit_template(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  prob_fun,
  hyper_set,
  hyper_name = NULL,
```

```
  loss_output_types = "recv",
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  save_model = FALSE,
  is_virtual_run = FALSE,
  loss = "density_ratio_nll",
  eps = 1e-12,
  ...
)
```

Arguments

`data`	A matrix-like object accepted by `prob_fun`, with observations in rows.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval model.
`l_end`, `r_end`	Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.
`prob_fun`	A function called with `data`, binary labels `y`, `train_id`, `eval_id`, one hyperparameter value `hyper`, and its index `hyper_id`. It must return one class-1 probability per evaluation row. Probabilities must be calibrated to the empirical class proportion in `y[train_id]`. If fitting uses class weights or class-balanced sampling, recalibrate the probabilities to that empirical training prior before returning them.
`hyper_set`	A vector, list, matrix, or data frame of candidate hyperparameters. A vector or list uses one element per candidate; a matrix or data frame uses one row per candidate.
`hyper_name`	`NULL` or a non-empty character scalar naming the tuning parameter, such as `"lambda"`. It is carried in the loss-output metadata so summaries and tuning-path plots can use a model-specific label. Use `NULL` when `hyper_set` is not described by one common name.
`loss_output_types`	A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.
`nfolds`	An integer scalar of at least 2 giving the number of global `"op"` folds. The fitted interval must contain at least `nfolds` observations and span at least two fold labels.
`fold_type`	A character scalar selecting fold construction. Classifier cross-fitting fits against the full data and supports only the globally anchored, interlaced `"op"` construction.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` Unused for classifier cross-fitting, which supports only `"op"` folds. Retained for argument compatibility.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

`is_virtual_run` A logical scalar used as a metadata-query flag by `reliever_generic()`. When `TRUE`, return only the number and metadata of loss outputs without fitting any model.

`loss` A character scalar selecting the segment loss computed from predicted probabilities. The supported value `"density_ratio_nll"` converts class probabilities to density-ratio negative log-likelihood. The conversion corrects for the inside-interval class proportion among the fold's training rows. If p is the predicted probability, π_t that training proportion, and $\pi = (r - l + 1)/n$, it sets $q = \{p/(1-p)\}\{(1-\pi_t)/\pi_t\}$ and returns $-\log[q/\{\pi q + (1-\pi)\}]$.

`eps` Probability-clipping constant in `[.Machine$double.eps, 0.5)`. Clipping precedes logarithms and keeps both endpoints distinct from zero and one in double precision.

`...` Additional arguments passed to `prob_fun`.

Value

A named list. With `is_virtual_run = TRUE`, it contains an integer scalar `n_loss_outputs` and a data frame `loss_output_meta`. Otherwise, it contains the requested numeric observation-by-output `loss` matrix and `model = NULL`. See `reg_fun_crossfit_template()` for the full return convention and column definitions.

See Also

`reg_fun_crossfit_template()`, `reg_fun_ranger_crossfit()`, and `reg_fun_mlp_crossfit()`.

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
prob_fun <- function(data, y, train_id, eval_id, hyper, ...) {
  rep(pmin(pmax(mean(y[train_id]) + hyper$offset, 0.01), 0.99),
      length(eval_id))
```

```
}
out <- reg_fun_clf_crossfit_template(
  data = x, l = 241, r = 660, l_end = 211, r_end = 690,
  prob_fun = prob_fun, hyper_set = data.frame(offset = c(0, 0.05)),
  nfolds = 2
)
dim(out$loss)
```

`reg_fun_crossfit_template` *Generic cross-fitted interval-loss template*

Description

Turn a model-specific observation-loss function into a complete crossfit output set. Users implement only `loss_fun`: fit the model on `train_id` and return the loss of every row in `eval_id`, once for each candidate hyperparameter. The template constructs folds, combines their losses, and returns the requested recycled-CV (ReCV) outputs. By default only the interval-adaptive `recv` loss is returned.

Usage

```
reg_fun_crossfit_template(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  loss_fun,
  hyper_set,
  hyper_name = NULL,
  loss_output_types = "recv",
  nfolds = 5,
  fold_type = c("op", "blk", "stable_blk"),
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  save_model = FALSE,
  is_virtual_run = FALSE,
  ...
)
```

Arguments

`data`	A matrix-like object accepted by `loss_fun`, with observations in rows.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval model.
`l_end`, `r_end`	Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.

`loss_fun` A function called with `data`, `train_id`, `eval_id`, `hyper_set`, the four interval endpoints, and `n_total`. It must return a numeric matrix with one row per `eval_id` and one column per hyperparameter. Every column must use the same observation-level scoring rule, units, normalization, and fold weighting. With one hyperparameter, a numeric vector is also accepted.

`hyper_set` A vector, list, matrix, or data frame of candidate hyperparameters. A vector or list uses one element per candidate; a matrix or data frame uses one row per candidate.

`hyper_name` `NULL` or a non-empty character scalar naming the tuning parameter, such as `"lambda"`. It is carried in the loss-output metadata so summaries and tuning-path plots can use a model-specific label. Use `NULL` when `hyper_set` is not described by one common name.

`loss_output_types`
A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`nfolds` An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.

`fold_type` A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const`
A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

is_virtual_run	A logical scalar used as a metadata-query flag by reliever_generic(). When TRUE, return only the number and metadata of loss outputs without fitting any model.
...	Additional arguments passed to loss_fun.

Value

A named list following the custom reg_fun calling convention documented in the *Writing a custom reg_fun* section of reliever_generic(). With is_virtual_run = TRUE, the list contains the integer scalar n_loss_outputs and the data frame loss_output_meta. Otherwise, it contains the numeric observation-by-output matrix loss and model = NULL. According to loss_output_types, the matrix can contain:

- recv: within each fitted interval, choose the hyperparameter with the smallest cross-fitted loss and return that cross-fitted loss.
- incv: use the same interval-specific hyperparameter chosen by recv, but return its in-sample loss.
- crossfit_homo_hyper: one cross-fitted-loss output for every hyperparameter, keeping that value fixed rather than choosing it separately within each interval.

Selection follows two workflows:

- Interval-adaptive ReCV uses select_by_run() with run type "recv" and criterion "loss".
- Homogeneous-hyperparameter ReCV chooses K and one globally fixed hyperparameter from stored fixed-hyperparameter rows.

Apply select_across_runs() for the homogeneous workflow.

Its run type is "crossfit_homo_hyper"; request those rows first.

See Also

reliever_generic(), reg_fun_clf_crossfit_template(), and select_by_run().

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
loss_fun <- function(data, train_id, eval_id, hyper_set, ...) {
  center <- colMeans(data[train_id, , drop = FALSE])
  vapply(hyper_set, function(shrinkage) {
    fitted_center <- shrinkage * center
    rowMeans(
      sweep(data[eval_id, , drop = FALSE], 2, fitted_center, "-")^2
    )
  }, numeric(length(eval_id)))
}
out <- reg_fun_crossfit_template(
  data = x, l = 241, r = 660, l_end = 211, r_end = 690,
  loss_fun = loss_fun, hyper_set = c(0.8, 1),
  nfolds = 2
```

```
)
dim(out$loss)
```

`reg_fun_em` *Exponential-family interval loss*

Description

Estimate one distribution on rows `l:r` and return observation-level negative twice log-likelihood on rows `l_end:r_end`. Supported family names are grouped as follows:

- `"binom"`, `"multinom"`, and `"pois"`;
- `"exp"`, `"geom"`, and `"diri"`;
- `"gamma"`, `"beta"`, `"chisq"`, and `"invgauss"`.

Full distribution names are also accepted; see `family` below.

Usage

```
reg_fun_em(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  family,
  size = NULL
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`l`, `r`	Inclusive fitting-interval endpoints.
`l_end`, `r_end`	Inclusive evaluation-interval endpoints.
`save_model`	Retain fitted parameters in the returned `model`.
`is_virtual_run`	Return loss-output metadata without fitting.
`family`	Exponential-family distribution name.
`size`	Number of trials, required for binomial and multinomial data.

Details

This is the low-level interval loss used by `reliever_em()`; most users should call that wrapper.

Value

A list containing the observation-level `loss` matrix and, when requested, the fitted `model`.

See Also

reliever_em(), reg_fun_glm()

Examples

```
set.seed(2026)
x <- rexp(30, rate = 2)
out <- reg_fun_em(
  x, l = 1, r = 20, l_end = 21, r_end = 30,
  family = "exp", save_model = TRUE
)
head(out$loss)
out$model$param
```

---

reg_fun_glm *Generalized-linear-model interval loss*

Description

Fit `stats::glm.fit()` on rows `l:r` and return the selected family's observation-level deviance residuals for rows `l_end:r_end`. The leading `response_ncol` columns are the response and the remaining columns are predictors. For grouped binomial data, provide successes and failures in the first two columns and use `response_ncol = 2`.

Usage

```
reg_fun_glm(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  family = stats::gaussian(),
  intercept = TRUE,
  response_ncol = 1L
)
```

Arguments

`data`	Numeric matrix with the response column or columns first and predictors afterward.
`l`, `r`	Inclusive fitting-interval endpoints.
`l_end`, `r_end`	Inclusive evaluation-interval endpoints.
`save_model`	Retain fitted parameters in the returned `model`.
`is_virtual_run`	Return loss-output metadata without fitting.
`family`	A GLM family object, family function, or the name of a `stats` family function.
`intercept`	Add an intercept to the interval model.
`response_ncol`	Number of leading response columns; use 2 for binomial successes and failures.

Details

This is the low-level interval loss used by `reliever_glm()`; most users should call that wrapper.

Value

A list containing the observation-level `loss` matrix and, when requested, the fitted `model`.

See Also

`reliever_glm()`, `reg_fun_lm()`

Examples

```
set.seed(2026)
x <- rnorm(30)
trials <- rep(5L, 30)
success <- rbinom(30, trials, plogis(x))
out <- reg_fun_glm(
  cbind(success, trials - success, x),
  l = 1, r = 20, l_end = 21, r_end = 30,
  family = binomial(), response_ncol = 2, save_model = TRUE
)
head(out$loss)
out$model$coefficients
```

---

reg_fun_kde_l2 *Empirical-grid KDE L2 interval loss*

Description

This function evaluates a kernel-density L_2 loss on a fixed common grid. Let $z_1, \ldots, z_m$ be that grid and let $G_{ij} = k_h(X_i, z_j)$ be row i of `data`. For a segment I, its KDE on the grid is

$$\widehat{f}_I(z_j) = |I|^{-1} \sum_{i \in I} G_{ij}.$$

The observation-level interval loss returned for row i is

$$\ell_i(I) = m^{-1} \sum_{j=1}^{m} \{G_{ij} - \widehat{f}_I(z_j)\}^2.$$

Summing these losses over the evaluation rows gives the empirical-grid discretization of the KDE L_2 criterion used for changepoint detection.

Usage

```
reg_fun_kde_l2(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE
)
```

Arguments

`data`	Numeric matrix whose rows are kernel-density contributions evaluated at common locations. A numeric vector is treated as a one-column matrix. A generic fixed feature matrix is also accepted as a computational extension.
`l`, `r`	Rows used to estimate the segment KDE.
`l_end`, `r_end`	Rows whose losses are returned.
`save_model`	Return the fitted segment KDE as `model = list(center = ...)`.
`is_virtual_run`	Query flag used by `reliever_generic()`. When `TRUE`, report one loss output without fitting.

Details

With a Gram matrix K, the common evaluation locations are the observed sample points and $K_{ij} = k_h(X_i, X_j)$. The matrix is computed once and reused to construct each segment KDE and evaluate its L_2 loss; it is a computational representation rather than the statistical target. A generic precomputed feature matrix is also accepted; it represents KDE L2 when its columns are evaluations of one fixed density kernel on a common grid. Omitting a normalizing constant shared by that fixed kernel rescales all candidate losses by the same positive factor.

The kernel and bandwidth are fixed for this loss. For data-driven bandwidth selection, use the `"kde_nll_crossfit"` family. For fixed KDE-L2, use the `"kde_l2"` family or `reliever_kde_l2()`. KDE-NLL constructs and reuses a bandwidth-independent pairwise distance matrix. KDE-L2 instead constructs one fixed kernel-evaluation matrix at the supplied bandwidth.

Value

With `is_virtual_run = TRUE`, metadata for one KDE-L2 loss output. Otherwise, a list containing a one-column `loss` matrix and `model`. When `save_model = TRUE`, `model` is `list(center = ...)`, where `center` contains the fitted segment KDE values on the common grid, or the segment mean for a generic fixed-feature extension; otherwise it is `NULL`. See the *Writing a custom reg_fun* section of `reliever_generic()` for the common return convention.

References

Padilla, O. H. M., Yu, Y., Wang, D., and Rinaldo, A. (2021). Optimal nonparametric multivariate change point detection and localization. *IEEE Transactions on Information Theory*, 68(3), 1922–1944.

See Also

`reliever_kde_l2()`, `reliever_kde_nll_crossfit()`, `reliever_generic()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
dist_sq <- as.matrix(dist(x))^2
bandwidth <- 1
```

```
kernel_mat <- exp(-dist_sq / (2 * bandwidth^2))
out <- reg_fun_kde_l2(kernel_mat, 241, 660, 211, 690)
dim(out$loss)
```

`reg_fun_kde_nll_crossfit` *Cross-fitted KDE negative-log-likelihood interval loss*

Description

Construct interval-level cross-fitted KDE negative-log-likelihood losses for a path of bandwidths and one fixed isotropic density kernel. The low-level input is a squared-distance matrix; each fold estimates density from its training rows and evaluates held-out rows. Most users should pass the original observations to `reliever(X, cpd_family = "kde_nll_crossfit")`. Supplying a precomputed squared-distance matrix together with `var_dim` remains supported.

Usage

```
reg_fun_kde_nll_crossfit(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  nfolds = 5,
  fold_type = c("op", "blk", "stable_blk"),
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  save_model = FALSE,
  is_virtual_run = FALSE,
  bandwidth_vec = NULL,
  var_dim = NULL,
  kernel = "gaussian",
  kernel_args = list(),
  distance_power = 2L
)
```

Arguments

`data`	Numeric n by n matrix of pairwise squared distances.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval KDE.
`l_end`, `r_end`	Integer scalars giving the first and last rows whose negative log densities are returned.
`nfolds`	An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.

`fold_type` A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.

`loss_output_types` A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. KDE models are not retained.

`is_virtual_run` A logical scalar used as a metadata-query flag. When `TRUE`, return only the number and metadata of loss outputs.

`bandwidth_vec` `NULL` or a numeric vector of positive KDE bandwidths. `NULL` uses the scale-adaptive default grid.

`var_dim` `NULL` or a positive integer scalar giving the dimension of the original observations. It is required when it cannot be inferred.

`kernel` A character scalar selecting `"gaussian"`, `"laplace"`, or `"student"`.

`kernel_args` A named list of kernel-specific settings.

`distance_power` An integer scalar equal to 1 or 2 identifying whether `data` stores distances or squared distances.

Value

A named list. With `is_virtual_run = TRUE`, it contains the integer scalar `n_loss_outputs` and data frame `loss_output_meta`. Otherwise, it contains the requested numeric observation-by-output `loss` matrix and `model = NULL`. See `reg_fun_crossfit_template()` for the full return convention and column definitions.

See Also

reliever_kde_nll_crossfit() and reg_fun_crossfit_template().

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
dist_sq <- as.matrix(dist(x))^2
out <- reg_fun_kde_nll_crossfit(
  data = dist_sq, l = 241, r = 660, l_end = 211, r_end = 690,
  var_dim = 5, nfolds = 2
)
colSums(out$loss)
```

---

reg_fun_kde_nll_solpath

Radial-kernel KDE negative-log-likelihood interval loss

Description

Fit an isotropic kernel density estimate on rows `l:r` and return the negative log density of rows `l_end:r_end`, once for each bandwidth in `bandwidth_vec`. The low-level input is an n by n squared-Euclidean-distance matrix, so pairwise distances are computed only once. Most users can instead pass the original observations to reliever().

Usage

```
reg_fun_kde_nll_solpath(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  bandwidth_vec = NULL,
  var_dim = NULL,
  kernel = "gaussian",
  kernel_args = list(),
  distance_power = 2L
)
```

Arguments

`data`	Square squared-distance matrix.
`l, r`	Rows used to fit the interval KDE.

l_end, r_end	Rows whose negative log densities are returned.
save_model	Accepted so this function can be used as a reg_fun; KDE models are not retained.
is_virtual_run	Query flag used by reliever_generic(). When TRUE, return only the number and metadata of loss outputs.
bandwidth_vec	Positive KDE bandwidths. NULL uses the scale-adaptive grid documented in reliever_kde_nll().
var_dim	Dimension of the original observations used to construct the squared distances. It appears in the density normalization constant and must be a positive integer because it cannot be recovered from a distance matrix.
kernel	Isotropic density kernel: "gaussian", "laplace", or "student".
kernel_args	Named list of kernel-specific settings; Student accepts positive df (default 5).
distance_power	Advanced representation flag. The ordinary value 2 means that data contains squared distances. The high-level Laplace wrapper internally uses 1 after taking the square root once for the whole data set; users normally should not change this argument.

Details

In dimension p, Gaussian uses the normalized density

$$(2\pi)^{-p/2}h^{-p}\exp\{-r^2/(2h^2)\},$$

radial Laplace uses

$$\frac{\Gamma(p/2)}{2\pi^{p/2}\Gamma(p)}h^{-p}\exp\{-r/h\},$$

and Student with degrees of freedom ν uses

$$\frac{\Gamma((\nu+p)/2)}{\Gamma(\nu/2)(\nu\pi)^{p/2}h^p}\{1+r^2/(\nu h^2)\}^{-(\nu+p)/2},$$

where $r = \|x-y\|_2$. All calculations use row-wise log-sum-exp rather than materializing one full kernel matrix per bandwidth. One Euclidean-distance matrix can therefore be reused for any isotropic radial kernel. Anisotropic, coordinate-product, or observation-adaptive kernels require a different representation or a custom reg_fun.

Value

With is_virtual_run = TRUE, metadata linking each loss output to its bandwidth. Otherwise, a list containing an observation-by-bandwidth loss matrix and model = NULL. See the *Writing a custom reg_fun* section of reliever_generic() for the common return convention.

References

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint Detection in Complex Models: Cross-Fitting Is Needed. arXiv:2411.07874.

See Also

reliever_kde_nll(), reg_fun_kde_nll_crossfit(), and reliever_generic().

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
dist_sq <- as.matrix(dist(x))^2
out <- reg_fun_kde_nll_solpath(dist_sq, 241, 660, 211, 690,
                       var_dim = 5)
dim(out$loss)
```

---

reg_fun_lasso_crossfit

Cross-fitted lasso interval loss

Description

Fit a glmnet lasso path within each interval fold. The interval-adaptive `recv` column chooses a normalized lambda setting in each fitted interval. When requested, the `crossfit_homo_hyper` columns support joint selection of one normalized setting and K. Most users should call `reliever()` with `cpd_family = "lasso_crossfit"`. The built-in model uses `intercept = FALSE` and `standardize = FALSE`; predictors should already be on their intended scale.

Usage

```
reg_fun_lasso_crossfit(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  nfolds = 5,
  fold_type = c("op", "blk", "stable_blk"),
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  save_model = FALSE,
  is_virtual_run = FALSE,
  lam_set = NULL,
  family = "gaussian",
  thresh = 1e-07
)
```

Arguments

`data`	Numeric matrix with the response in the first column and at least two predictor columns, as required by `glmnet`.
`l, r`	Integer scalars giving the first and last rows used to fit the interval model.

`l_end`, `r_end` Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.

`nfolds` An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.

`fold_type` A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const`
A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.

`loss_output_types`
A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

`is_virtual_run` A logical scalar used as a metadata-query flag by `reliever_generic()`. When `TRUE`, return only the number and metadata of loss outputs without fitting any model.

`lam_set` A numeric vector of finite positive lambda values on a sample-size-independent scale. A fold containing m training observations passes `lam_set / sqrt(m)` to glmnet.

`family` A character scalar selecting the response family: `"gaussian"`, `"binomial"`, or `"poisson"`. Gaussian returns squared prediction loss; binomial and Poisson return negative log-likelihood loss. Binomial responses must be 0 or 1; Poisson responses may contain any finite non-negative values.

`thresh` Positive convergence threshold passed to `glmnet`.

Value

A named list. With `is_virtual_run = TRUE`, it contains the integer scalar `n_loss_outputs` and data frame `loss_output_meta`. Otherwise, it contains the requested numeric observation-by-output `loss` matrix and `model = NULL`. See `reg_fun_crossfit_template()` for the full return convention and column definitions.

See Also

`reliever_lasso_crossfit()`, `reg_fun_lasso_solpath()`, `reg_fun_crossfit_template()`

Examples

```
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
data <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
lam_set <- 0.1 * 1.23^(30:0)
out <- reg_fun_lasso_crossfit(
  data = data, l = 241, r = 660, l_end = 211, r_end = 690,
  lam_set = lam_set, nfolds = 2
)
colSums(out$loss)
```

`reg_fun_lasso_solpath` *Lasso solution-path interval loss*

Description

Fit one glmnet lasso solution path on rows `l:r`, then return the observation-level prediction loss on `l_end:r_end` for every lambda. This is the low-level loss function used by `reliever()` when `cpd_family = "lasso"`, and by outer-CV lasso workflows. The returned columns define candidates; an in-sample minimum must not be used to select lambda. Use cross-fitted loss, outer CV, or independent hold-out loss for that choice. For an exact fitted interval, training loss ordinarily decreases as lambda decreases, so an interior minimum is neither expected nor a valid grid check. A fixed-K held-out or cross-fitted loss profile should instead attain its minimum away from both ends of a sufficiently wide lambda grid. The built-in model uses `intercept = FALSE` and `standardize = FALSE`; predictors should already be on their intended scale.

Usage

```
reg_fun_lasso_solpath(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  lam_set = NULL,
```

```
  family = "gaussian",
  thresh = 1e-07
)
```

Arguments

`data`	Numeric matrix with response in the first column and predictors in the remaining columns. At least two predictor columns are required by `glmnet`.
`l`, `r`	Rows used to fit the lasso path.
`l_end`, `r_end`	Rows for which prediction losses are returned.
`save_model`	Return the fitted glmnet model.
`is_virtual_run`	Query flag used by `reliever_generic()`. When `TRUE`, return lambda metadata without fitting glmnet.
`lam_set`	Finite positive values on a sample-size-independent scale. On an interval containing m observations, the lambda passed to glmnet is `lam_set / sqrt(m)`. This low-level function requires an explicit path; `reliever(X, y, cpd_family = "lasso")` constructs one automatically when omitted.
`family`	Response family: `"gaussian"`, `"binomial"`, or `"poisson"`. Gaussian returns squared prediction loss; binomial and Poisson return negative log-likelihood loss. Binomial responses must be 0 or 1; Poisson responses may contain any finite non-negative values.
`thresh`	Convergence threshold passed to `glmnet::glmnet()`.

Value

With `is_virtual_run = TRUE`, metadata linking each loss output to its lambda. Otherwise, a list with:

`loss` An observation-by-lambda matrix.

`model` The fitted glmnet path when `save_model = TRUE`; otherwise `NULL`.

See Also

`reliever()` and `reg_fun_lasso_crossfit()`.

Examples

```
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
data <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
lam_set <- 0.1 * 1.23^(30:0)
out <- reg_fun_lasso_solpath(data, 1, 300, 1, 360, lam_set = lam_set)
dim(out$loss)
```

`reg_fun_lm` *Linear-model interval loss*

Description

Fit ordinary least squares on rows `l:r` and return squared residuals for rows `l_end:r_end`. The first column of `data` is the response and all remaining columns are predictors.

Usage

```
reg_fun_lm(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  intercept = TRUE
)
```

Arguments

`data`	Numeric matrix with the response first and predictors afterward.
`l`, `r`	Inclusive fitting-interval endpoints.
`l_end`, `r_end`	Inclusive evaluation-interval endpoints.
`save_model`	Retain fitted parameters in the returned `model`.
`is_virtual_run`	Return loss-output metadata without fitting.
`intercept`	Add an intercept to the interval model.

Details

This is the low-level interval loss used by `reliever_lm()`; most users should call that wrapper.

Value

A list containing the observation-level `loss` matrix and, when requested, the fitted `model`.

See Also

`reliever_lm()`, `reg_fun_glm()`

Examples

```
set.seed(2026)
x <- matrix(rnorm(60), ncol = 2)
y <- 1 + x[, 1] - x[, 2] + rnorm(30, sd = 0.3)
out <- reg_fun_lm(
  cbind(y, x), l = 1, r = 20, l_end = 21, r_end = 30,
  save_model = TRUE
)
```

```
head(out$loss)
out$model$coefficients
```

`reg_fun_mean` *Mean-square interval-loss function*

Description

Estimate the segment mean from rows `l:r` and return one mean-square loss per row in `l_end:r_end`. For multivariate observations, each loss is the average squared error across columns. This R interval-loss function is intended for `reliever_generic()` and `cv.reliever_generic()`. For ordinary mean-change detection without outer cross-validation, `reliever(X, cpd_family = "mean")` dispatches to the faster native C++ implementation.

Usage

```
reg_fun_mean(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`l, r`	Rows used to estimate the segment mean.
`l_end, r_end`	Rows whose mean-square losses are returned.
`save_model`	Return the fitted segment mean as `list(center = ...)` when `TRUE`.
`is_virtual_run`	Query flag used by `reliever_generic()`. When `TRUE`, return metadata for the single loss output without estimating a mean.

Value

With `is_virtual_run = TRUE`, metadata for one loss output. Otherwise, a list containing a one-column `loss` matrix and `model`. When `save_model = TRUE`, `model` is `list(center = ...)`; otherwise it is `NULL`. See the *Writing a custom reg_fun* section of `reliever_generic()` for the common return convention.

See Also

`reliever_mean()`, `cv.reliever_generic()`, `reliever_generic()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
out <- reg_fun_mean(x, l = 241, r = 660, l_end = 211, r_end = 690)
dim(out$loss)
```

---

reg_fun_meanvar *Simultaneous mean-and-covariance interval loss*

Description

Estimate both the mean and maximum-likelihood covariance matrix on rows `l:r`, then return observation-level negative twice multivariate Gaussian log-likelihood on rows `l_end:r_end`.

Usage

```
reg_fun_meanvar(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`l, r`	Inclusive fitting-interval endpoints.
`l_end, r_end`	Inclusive evaluation-interval endpoints.
`save_model`	Retain fitted parameters in the returned `model`.
`is_virtual_run`	Return loss-output metadata without fitting.

Details

This is the low-level interval loss used by `reliever_meanvar()`; most users should call that wrapper.

Value

A list containing the observation-level `loss` matrix and, when requested, the fitted `model`.

See Also

`reliever_meanvar()`, `reg_fun_var()`

Examples

```
set.seed(2026)
x <- matrix(rnorm(60), ncol = 2)
out <- reg_fun_meanvar(
  x, l = 1, r = 20, l_end = 21, r_end = 30, save_model = TRUE
)
head(out$loss)
out$model$center
```

---

reg_fun_mean_crossfit *Cross-fitted mean-square interval loss*

Description

Construct interval-level cross-fitted and in-sample mean-square losses. In each fold, the segment mean is estimated from the training observations and squared error is returned for held-out observations. Most users should call `reliever(X, cpd_family = "mean_crossfit")`; this function is the lower-level loss function for `reliever_generic()` and custom extensions.

Usage

```
reg_fun_mean_crossfit(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  nfolds = 5,
  fold_type = c("op", "blk", "stable_blk"),
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  save_model = FALSE,
  is_virtual_run = FALSE
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval model.
`l_end`, `r_end`	Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.
`nfolds`	An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.
`fold_type`	A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.

`loss_output_types` A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

`is_virtual_run` A logical scalar used as a metadata-query flag by `reliever_generic()`. When `TRUE`, return only the number and metadata of loss outputs without fitting any model.

Value

A named list. With `is_virtual_run = TRUE`, it contains the integer scalar `n_loss_outputs` and data frame `loss_output_meta`. Otherwise, it contains a numeric observation-by-output `loss` matrix and `model = NULL`. The default output is `recv`, the cross-fitted mean-square loss; `incv` can additionally return its in-sample counterpart. See `reg_fun_crossfit_template()` for the full return convention.

See Also

`reliever_mean_crossfit()`, `reliever_mean()`, `reg_fun_crossfit_template()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, mean = 0, sd = 0.5),
  rnorm(n_seg, mean = 4, sd = 0.5),
```

```
  rnorm(n_seg, mean = -4, sd = 0.5)
)
out <- reg_fun_mean_crossfit(
  data = x, l = 241, r = 660, l_end = 211, r_end = 690, nfolds = 2
)
colSums(out$loss)
```

`reg_fun_mlp_crossfit` *Cross-fitted MLP-classifier interval loss*

Description

Use `nnet::nnet()` as the probability model inside `reg_fun_clf_crossfit_template()`. The classifier distinguishes observations inside the fitted interval from observations outside it; its probabilities are converted to density-ratio losses. This function requires the optional `nnet` package. Most users should call `reliever(X, cpd_family = "mlp_crossfit")`.

Usage

```
reg_fun_mlp_crossfit(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  save_model = FALSE,
  is_virtual_run = FALSE,
  hyper_set = data.frame(size = 8),
  nnet_args = list()
)
```

Arguments

`data`	Numeric matrix or data frame with observations in rows and predictor features in columns.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval model.
`l_end`, `r_end`	Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.
`nfolds`	An integer scalar of at least 2 giving the number of global `"op"` folds. The fitted interval must contain at least `nfolds` observations and span at least two fold labels.
`fold_type`	A character scalar selecting fold construction. Classifier cross-fitting fits against the full data and supports only the globally anchored, interlaced `"op"` construction.

op_size — A positive integer scalar giving, for fold_type = "op", the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

buffer_lag — A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at floor(n_core / (2 * nfolds)) so that every fold remains trainable; it can therefore be 0 on short intervals.

fold_stable_const — Unused for classifier cross-fitting, which supports only "op" folds. Retained for argument compatibility.

loss_output_types — A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default "recv" returns only the interval-adaptive cross-fitted loss. Use c("recv", "incv") to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include "crossfit_homo_hyper" to additionally return one cross-fitted output per fixed hyperparameter for subsequent select_across_runs() selection. "recv" is required; an input that omits it produces an error. Returned columns always follow the order recv, incv when requested, then crossfit_homo_hyper in hyper_set order.

save_model — A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

is_virtual_run — A logical scalar used as a metadata-query flag by reliever_generic(). When TRUE, return only the number and metadata of loss outputs without fitting any model.

hyper_set — A data frame, matrix, or list of candidate MLP settings. Supply a data frame or matrix with one row per setting and uniquely named columns, or a list whose elements are named argument lists.

nnet_args — A named list of fixed nnet::nnet() arguments shared by every candidate in hyper_set. Observation weights are rejected because the built-in probability conversion requires the empirical training class proportion. The wrapper manages x and y; do not include them in either argument list. Weighted models require the generic classifier template with recalibrated probabilities. Supply each argument in only one of hyper_set and nnet_args.

Value

A named list. With is_virtual_run = TRUE, it contains the integer scalar n_loss_outputs and data frame loss_output_meta. Otherwise, it contains the requested numeric observation-by-output loss matrix and model = NULL. See reg_fun_crossfit_template() for the full return convention and column definitions.

See Also

reliever_mlp_crossfit() and reg_fun_clf_crossfit_template().

Examples

```
if (requireNamespace("nnet", quietly = TRUE)) {
  set.seed(2026)
  n_seg <- 300
  x <- rbind(
    matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
  )
  out <- suppressWarnings(reg_fun_mlp_crossfit(
    data = x, l = 241, r = 660, l_end = 211, r_end = 690,
    hyper_set = data.frame(size = 2, maxit = 30),
    nfolds = 2
  ))
  colSums(out$loss)
}
```

---

reg_fun_nmcd *Univariate nonparametric CDF interval loss*

Description

Fit the empirical distribution on rows `l:r` and score each row in `l_end:r_end` by aggregating binary log loss across empirical-CDF cutpoints. Because it models the full univariate distribution rather than only its mean, the loss can detect changes in location, scale, or shape. The calculation is implemented in C++. For changepoint fitting, most users should call `reliever(x, cpd_family = "nmcd")`; this function is the lower-level interval-loss implementation.

Usage

```
reg_fun_nmcd(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  w_trunc = 0,
  sort_X = NULL
)
```

Arguments

`data`	Numeric vector, or one-column matrix, of observations.
`l, r`	Rows used to estimate the interval distribution.
`l_end, r_end`	Rows whose distribution losses are returned.
`save_model`	Accepted so this function can be used as a `reg_fun`; no fitted object is retained.
`is_virtual_run`	Query flag used by `reliever_generic()`. When `TRUE`, report one loss output without fitting.

`w_trunc` Fraction removed from each tail of the ordered full-sample CDF cutpoints. If the reference vector has length m, `floor(w_trunc * m)` cutpoints are omitted from both the lower and upper tails. The default 0 keeps every cutpoint used by the criterion.

`sort_X` Optional sorted reference vector that defines the empirical-CDF cutpoints. The default sorts the training observations in `data`; externally appended evaluation rows are excluded automatically. Model wrappers may resolve this reference once and reuse it.

Details

Let $z_1, \ldots, z_{m-1}$ be the ordered reference cutpoints, $\widehat{F}_I$ the empirical CDF fitted on interval I, and $B_{ij} = 1\{X_i \le z_j\}$. Apart from optional tail truncation, the observation loss is

$$-\sum_{j=1}^{m-1} \frac{m}{j(m-j)} [B_{ij} \log \widehat{F}_I(z_j) + (1 - B_{ij}) \log\{1 - \widehat{F}_I(z_j)\}].$$

Fitted probabilities equal to 0 or 1 are replaced by $1/(2|I|)$ or $1 - 1/(2|I|)$. This is a Reliever-compatible weighted empirical-CDF likelihood based on the NMCD construction.

Value

With `is_virtual_run = TRUE`, metadata for one additive empirical-CDF negative-log-likelihood output. Otherwise, a list containing a one-column `loss` matrix and `model = NULL`. See the *Writing a custom reg_fun* section of `reliever_generic()` for the common return convention.

References

Zou, C., Yin, G., Feng, L., and Wang, Z. (2014). Nonparametric maximum likelihood approach to multiple change-point problems. *The Annals of Statistics*, 42(3), 970–1002. DOI: 10.1214/14-AOS1210.

See Also

`reliever_nmcd()`, `reliever_generic()`, `reg_fun_kde_l2()`, `reg_fun_kde_nll_solpath()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, mean = 0, sd = 0.2),
  rnorm(n_seg, mean = 4, sd = 0.2),
  rnorm(n_seg, mean = -4, sd = 0.2)
)
out <- reg_fun_nmcd(x, 241, 660, 211, 690)
dim(out$loss)
```

`reg_fun_ranger_crossfit` *Cross-fitted ranger-classifier interval loss*

Description

Use `ranger::ranger()` as the probability model inside `reg_fun_clf_crossfit_template()`. The classifier distinguishes observations inside the fitted interval from observations outside it; its probabilities are converted to density-ratio losses. Most users should call `reliever(X, cpd_family = "ranger_crossfit")`.

Usage

```
reg_fun_ranger_crossfit(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  save_model = FALSE,
  is_virtual_run = FALSE,
  hyper_set = list(list()),
  ranger_args = list()
)
```

Arguments

`data`	Numeric matrix or data frame with observations in rows and predictor features in columns.
`l`, `r`	Integer scalars giving the first and last rows used to fit the interval model.
`l_end`, `r_end`	Integer scalars giving the first and last rows for which losses must be returned. This interval may extend beyond `l:r` because Reliever reuses one fitted model for nearby candidate intervals.
`nfolds`	An integer scalar of at least 2 giving the number of global `"op"` folds. The fitted interval must contain at least `nfolds` observations and span at least two fold labels.
`fold_type`	A character scalar selecting fold construction. Classifier cross-fitting fits against the full data and supports only the globally anchored, interlaced `"op"` construction.
`op_size`	A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` Unused for classifier cross-fitting, which supports only `"op"` folds. Retained for argument compatibility.

`loss_output_types` A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`save_model` A logical scalar accepted for compatibility with the interval-loss protocol. This template returns losses only and does not retain fitted models.

`is_virtual_run` A logical scalar used as a metadata-query flag by `reliever_generic()`. When `TRUE`, return only the number and metadata of loss outputs without fitting any model.

`hyper_set` A data frame, matrix, or list of candidate ranger settings. Supply a data frame or matrix with one row per setting and uniquely named columns, or a list whose elements are named argument lists.

`ranger_args` A named list of fixed `ranger::ranger()` arguments shared by every candidate in `hyper_set`. The built-in conversion rejects class weights, case weights, and class-specific sampling because it requires probabilities under the empirical training class proportion. The wrapper manages `formula`, `data`, `write.forest`, and `probability`; do not include them in either argument list. Other controls, including `num.threads`, may be supplied. Supply each argument in only one of `hyper_set` and `ranger_args`. Weighted classifiers require the generic classifier template with recalibrated probabilities.

Value

A named list. With `is_virtual_run = TRUE`, it contains the integer scalar `n_loss_outputs` and data frame `loss_output_meta`. Otherwise, it contains the requested numeric observation-by-output `loss` matrix and `model = NULL`. See `reg_fun_crossfit_template()` for the full return convention and column definitions.

References

Londschien, M., Bühlmann, P., and Kovács, S. (2023). Random forests for change point detection. *Journal of Machine Learning Research*, 24(216), 1–45.

See Also

`reliever_ranger_crossfit()` and `reg_fun_clf_crossfit_template()`.

Examples

```
if (requireNamespace("ranger", quietly = TRUE)) {
  set.seed(2026)
  n_seg <- 300
  x <- rbind(
    matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
  )
  out <- suppressWarnings(reg_fun_ranger_crossfit(
    data = x, l = 241, r = 660, l_end = 211, r_end = 690,
    hyper_set = data.frame(num.trees = 10, min.node.size = 5),
    ranger_args = list(seed = 2026),
    nfolds = 2
  ))
  colSums(out$loss)
}
```

---

reg_fun_var *Covariance-change interval loss*

Description

Fit one covariance matrix on rows `l:r`, using a fixed common mean. Return observation-level negative twice Gaussian log-likelihood on rows `l_end:r_end`. If `mu = NULL`, use the mean of all supplied data.

Usage

```
reg_fun_var(
  data,
  l,
  r,
  l_end = l,
  r_end = r,
  save_model = FALSE,
  is_virtual_run = FALSE,
  mu = NULL
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`l, r`	Inclusive fitting-interval endpoints.
`l_end, r_end`	Inclusive evaluation-interval endpoints.
`save_model`	Retain fitted parameters in the returned `model`.
`is_virtual_run`	Return loss-output metadata without fitting.
`mu`	Optional fixed common mean. A scalar is applied to every column.

Details

This is the low-level interval loss used by `reliever_var()`; most users should call that wrapper.

Value

A list containing the observation-level `loss` matrix and, when requested, the fitted `model`.

See Also

`reliever_var()`, `reg_fun_meanvar()`, `reg_fun_mean()`

Examples

```
set.seed(2026)
x <- matrix(rnorm(60), ncol = 2)
out <- reg_fun_var(
  x, l = 1, r = 20, l_end = 21, r_end = 30,
  mu = c(0, 0), save_model = TRUE
)
head(out$loss)
out$model$covariance
```

`reliever` *Reliever changepoint detection with built-in loss families*

Description

Primary fitting interface for built-in Reliever methods. `cpd_family = "mean"` fits mean changes by default; the alternatives cover regression, density, nonparametric, and classifier losses. `cpn_crit = "none"` retains the candidate path for later selection.

Usage

```
reliever(
  X,
  y = NULL,
  cpd_family = "mean",
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  dc_grid = NULL,
  ...
)
```

Arguments

X	A numeric vector, matrix, data frame, distance/kernel matrix, or family-specific data object required by the selected cpd_family. For regression families with y = NULL, the first column of X is treated as the response; lasso families require at least two predictor columns because they use glmnet. KDE-NLL families ordinarily accept the original observations and compute one reusable distance matrix; a precomputed squared-distance matrix is also accepted. KDE-L2 accepts one precomputed kernel-evaluation matrix, or raw observations together with one fixed kernel; it evaluates an empirical-grid kernel-density L2 loss. Mean, covariance, mean-and-covariance, exponential-family, NMCD, and classifier families accept observations in rows.
y	NULL, a numeric response vector, or for a grouped binomial GLM a two-column numeric matrix of successes and failures. Used by "lm", "glm", "lasso", and "lasso_crossfit". A grouped binomial GLM may use a two-column matrix of successes and failures. It is ignored with a warning for non-regression families.
cpd_family	A character scalar selecting the built-in loss family: • "mean": ordinary mean changes. • "mean_crossfit": cross-fitted mean changes. • "var" and "meanvar": covariance changes around a fixed common mean, or simultaneous mean and covariance changes, using multivariate Gaussian likelihood; see reliever_var() and reliever_meanvar(). • "lm" and "glm": changes in linear or generalized linear regression relationships; see reliever_lm() and reliever_glm(). • "em": changes in one selected exponential-family distribution. • "lasso" and "lasso_crossfit": regression-coefficient changes using an in-sample lasso path or cross-fitted lasso losses; see reliever_lasso() and reliever_lasso_crossfit(). Built-in lasso fits use no intercept and no automatic predictor standardization. • "kde_nll" and "kde_nll_crossfit": distribution changes using Gaussian, radial-Laplace, or multivariate-Student KDE negative log-likelihood; see reliever_kde_nll() and reliever_kde_nll_crossfit(). • "kde_l2": distributional changes under an empirical-grid kernel-density L2 loss with one fixed kernel. See reliever_kde_l2(). • "nmcd": arbitrary univariate distribution changes using an empirical-CDF loss; see reliever_nmcd(). • "ranger_crossfit": cross-fitted random-forest changes. • "mlp_crossfit": cross-fitted MLP classifier changes.
cpn_max	A non-negative integer scalar giving the maximum number of changepoints considered by fixed-K searches. Set it above the largest plausible changepoint count.
dm	A positive integer scalar giving the minimum segment length on the original sample scale. For a gridded search, Reliever uses a conservative grid-cell minimum that guarantees every returned segment contains at least dm observations.
cov_rate	A numeric scalar in (0, 1] giving the fraction of eligible intervals covered by the deterministic representative-interval collection, in (0, 1]. Use 1 for a full search.
method	A character scalar selecting the changepoint search algorithm. Choices are segment neighbourhood ("SN"), Reliever's globally greedy wild binary segmentation ("WBS"), recursive wild binary segmentation ("WBS_recursive"), seeded

binary segmentation (`"SeedBS"`), binary segmentation (`"BS"`), optimal partitioning (`"OP"`), and PELT (`"PELT"`). PELT applies pruning; OP runs the same penalized dynamic program without pruning.

`cpn_crit` A character scalar or non-negative numeric scalar specifying the optional rule used to select K while constructing the fit. Every candidate has a stored `loss` and changepoint count K:

- `"loss"` chooses the smallest stored loss without a K penalty. For recycled cross-validation, select the appropriate loss rows with `run_type` in a post-fit selector.
- `"aic"`, `"hqc"`, and `"sic"` minimize $loss + \gamma K$, with $\gamma = 2$, $2\log\log(n)$, and $\log(n)$, respectively. They apply directly to the stored loss, including RSS, and therefore retain its scale.
- `"rss_aic"`, `"rss_hqc"`, and `"rss_sic"` minimize $n\log(RSS/n)/2 + \gamma K$. This alternative log-RSS formulation may be preferable when residual variance is unknown or potentially heterogeneous.
- A non-negative number supplies γ directly in $loss + \gamma K$.
- `"none"`, the default, applies no loss-based K selection. For paths indexed directly by K, the compact summary is empty. For WBS thresholds or PELT/OP penalties, the summary still reports the segmentation associated with each supplied search value.

A criterion is applied separately within every selected loss path. Comparable paths can instead be selected jointly with `select_across_runs()`. In every formula, n is the original number of observations, including when a candidate grid compresses the search grid. The named presets penalize K only; they do not estimate model-family degrees of freedom. If `wbs_stop_crit` is supplied, selection compares only its mapped segmentations and the no-change baseline.

`pen_val` Non-negative numeric penalties for `"PELT"` or `"OP"`. Each value produces one candidate segmentation. Other methods ignore this argument.

`prune_value` A numeric scalar giving the constant in the PELT pruning inequality. The default 0 gives the standard PELT rule; `method = "OP"` disables pruning.

`M` A positive integer scalar giving the maximum number of random intervals retained by `"WBS"` and `"WBS_recursive"`.

`wbs_seed` `NULL` or an integer scalar used as the seed for random WBS interval generation.

`wbs_stop_crit` Optional numeric stopping thresholds for WBS-family methods. Larger thresholds tend to retain fewer changepoints. When supplied, selection compares their mapped segmentations and the no-change baseline.

`detail` A logical or character scalar. Use `TRUE` or `"cache"` to retain cache state and search-method diagnostics; use `FALSE` or `"none"` for a compact result. For non-native families the returned cache can also be supplied as `cache_profile` in a later compatible fit; the native mean cache is retained for inspection only.

`cache_backend` A character scalar selecting the cost-cache implementation used by every built-in family. `"by_loss_block"` stores losses for fitted representative intervals, while `"by_cost_mat"` stores reconstructed interval costs in an expanded matrix. A full search automatically uses `"by_cost_mat"`.

`owner_key` A logical scalar. With `cache_backend = "by_loss_block"`, cache the mapping from each exact interval to its representative interval. This usually improves speed at a modest memory cost.

`echo` A logical scalar indicating whether to print timing messages.

dc_grid_size	Regular candidate-grid spacing, or NULL to search every boundary. This coarse-grid option is motivated by the divide step of DCDP (Li, Wang, and Rinaldo, 2023); it does not apply the subsequent DCDP local-refinement step.
dc_grid	NULL or an increasing integer vector providing an advanced explicit interface for an irregular grid ending at the number of observations. Supply only one of dc_grid_size and dc_grid.
...	Model-specific arguments passed to the selected reg_fun; see that function's help page.

Details

Use cv.reliever() for CPSS outer-CV selection with its built-in single-path parametric losses or lasso path.

Cross-fitted changepoint families support two ReCV workflows:

- For interval-adaptive ReCV, use select_by_run() with run type "recv" and criterion "loss".
- Homogeneous-hyperparameter ReCV jointly selects one hyperparameter, K, and the changepoints from stored fixed-hyperparameter rows.

Apply select_across_runs() for the homogeneous workflow.

Its run type is "crossfit_homo_hyper"; request both crossfit outputs during fitting.

If these unpenalized-loss rules select too many changepoints, replace cpn_crit = "loss" with cpn_crit = "sic" to add a $\log(n)K$ penalty.

Information criteria are available through select_by_run(). Independent hold-out selection supports ordinary losses plus the built-in lasso- and KDE-NLL-crossfit scorers; other crossfit templates require a model-specific external scorer.

Focused reliever_*() functions expose the complete model-specific arguments for the same built-in methods. Use reliever_generic() for a custom interval-loss function.

Value

A list of class reliever_result with:

summary A data frame containing the compact selection shown when the object is printed and returned by summary(). With the ordinary cpn_crit = "none" default, this is empty for paths indexed directly by K. An explicit criterion selects K separately within each eligible loss output. A hyper_value identifies that row's model setting; it does not imply that the hyperparameter was compared across runs. For eligible loss outputs, supplied WBS thresholds or PELT/OP penalties are reported with their corresponding segmentations even when cpn_crit = "none".

cpd_path A list containing all distinct candidate segmentations. Its candidates table contains K, loss, and changepoint locations. For PELT/OP penalties or WBS stopping thresholds, selector_map records which supplied value produced each candidate.

run_meta A data frame containing metadata describing each searched loss output. It always links the exact numeric run_id to loss_output_id; built-in and custom losses may also declare row_type, a hyperparameter, and loss_kind. Post-fit selectors can match declared row_type labels through run_type, so ordinary ReCV workflows need not inspect numeric identifiers. Built-in loss_kind metadata describes the stored loss and can flag unlike kinds in across-run selection.

`settings` A named list containing the effective fitting configuration, including sample and search settings, cache controls, the resolved changepoint-number criterion, and model-specific loss arguments. Built-in response-based and kernel fits also record an `input_spec`. It validates later hold-out inputs and reconstructs pairwise matrices when the original kernel fit used raw data.

`timing` A data frame containing model-fitting counts and elapsed times by `run_id`.

`diagnostics` A list of method-specific details, such as WBS split gains or PELT pruning counts, when requested.

`cache_profile` A list of reusable cache objects when requested by `detail`.

Use `select_by_run()` for K selection within a stored loss path, `select_across_runs()` for a joint choice across comparable paths, `cv.reliever()` or `cv.reliever_generic()` for CPSS-style outer CV, and `select_holdout()` for compatible independent hold-out selection. The hold-out help page lists the supported crossfit families.

References

Qian, C., Wang, G., and Zou, C. (2025). Reliever: Relieving the burden of costly model fits for changepoint detection. *Journal of Machine Learning Research*, 26(203), 1–57.

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint detection in complex models: Cross-fitting is needed. arXiv:2411.07874.

Li, W., Wang, D., and Rinaldo, A. (2023). Divide and conquer dynamic programming: An almost linear time change point detection methodology in high dimensions. *Proceedings of Machine Learning Research*, 202, 20065–20148.

See Also

`reliever_generic()`, `cv.reliever()`, and `select_by_run()`.

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- reliever(
  X = x, cpn_max = 5, dm = 15, cov_rate = 0.6, method = "SN"
)
summary(fit) # empty: fitting did not choose K

# CPSS-style outer sample splitting selects K by held-out loss.
fit_cv <- cv.reliever(
  X = x, cpn_max = 5, dm = 15, cov_rate = 0.6, nfolds = 3
)
selected <- summary(fit_cv)
selected
stopifnot(identical(selected$K_hat, 2L))


# The model-based examples below take more than five seconds together.
# Regression-coefficient changes use the same entry point, with X and y
```

```
# supplied separately.
p <- 20
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
set.seed(2026)
reg_data <- dgp_linear_regression(
  n = 450, p = p, tau = c(150, 300),
  b0 = b0, delta = delta, sig = 1
)$data
reg_y <- reg_data[, 1]
reg_x <- reg_data[, -1, drop = FALSE]

# Cross-fitted lasso paths support both interval-adaptive ReCV and
# homogeneous-lambda ReCV without an additional outer-CV fit.
fit_lasso_cf <- reliever(
  X = reg_x, y = reg_y, cpd_family = "lasso_crossfit",
  cpn_max = 5, dm = 15, cov_rate = 0.7, method = "SN",
  nfolds = 2, nlambda = 25,
  loss_output_types = c("recv", "crossfit_homo_hyper")
)
selected_lasso_cf <- select_by_run(
  result = fit_lasso_cf, run_type = "recv", cpn_crit = "loss"
)
selected_lasso_cf
stopifnot(identical(selected_lasso_cf$K_hat, 2L))
selected_lasso_fixed <- select_across_runs(
  result = fit_lasso_cf,
  run_type = "crossfit_homo_hyper", cpn_crit = "loss"
)
selected_lasso_fixed
stopifnot(identical(selected_lasso_fixed$K_hat, 2L))
plot(fit_lasso_cf, run_type = "recv")
plot(
  fit_lasso_cf, x_axis = "hyperparameter", K = 2,
  run_type = "crossfit_homo_hyper", cpn_crit = "loss", log = "x"
)

# KDE-NLL also receives the original observations. One distance matrix is
# reused across the bandwidth path; no bandwidth-specific Gram path is kept.
x_kernel <- x[seq(1, nrow(x), by = 3), , drop = FALSE]
fit_kde_cf <- reliever(
  X = x_kernel, cpd_family = "kde_nll_crossfit",
  kernel = "laplace",
  cpn_max = 3, dm = 10, cov_rate = 0.6, method = "SN", nfolds = 2
)
selected_kde_cf <- select_by_run(
  result = fit_kde_cf, run_type = "recv", cpn_crit = "sic"
)
selected_kde_cf
stopifnot(identical(selected_kde_cf$K_hat, 2L))
# Multivariate Student KDE is selected with, for example,
# kernel = "student", kernel_args = list(df = 5).

# KDE-L2 evaluates a fixed density-kernel L2 loss on the common sample grid.
# One Gram matrix is constructed and reused; bandwidths are not compared.
fit_kernel_l2 <- reliever(
  X = x_kernel, cpd_family = "kde_l2",
```

```
  kernel = "gaussian", bandwidth = 1,
  cpn_max = 3, dm = 10, cov_rate = 0.6, method = "SN"
)
selected_kde_l2 <- select_by_run(
  result = fit_kernel_l2, cpn_crit = "rss_sic"
)
selected_kde_l2
stopifnot(identical(selected_kde_l2$K_hat, 2L))
```

`reliever_em` *Reliever for exponential-family distribution changes*

Description

Fit one distribution in each interval and use observation-level negative twice log-likelihood as the segment loss. Supported families are `"binom"`, `"multinom"`, `"pois"`, `"exp"`, `"geom"`, `"diri"`, `"gamma"`, `"beta"`, `"chisq"`, and `"invgauss"`.

Usage

```
reliever_em(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  family,
  size = NULL
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.

`detail`, `cache_backend`, `owner_key`, `echo`
: Common output and cache controls; see `reliever()`.

`dc_grid_size`, `dc_grid`
: Common candidate-grid controls; see `reliever()`.

`cache_profile`, `run_cpd_ids`
: Advanced reusable-cache and loss-output controls; see `reliever_generic()`.

`family`
: Exponential-family distribution name.

`size`
: Number of trials, required for binomial and multinomial data.

Details

The main built-in entry is `reliever()` with `cpd_family = "em"`. Use `cv.reliever()` with the same inputs to select K by outer sample-splitting.

Value

A `reliever_result` object.

See Also

`reg_fun_em()`, `cv.reliever()`

Examples

```
set.seed(2026)
x <- c(rexp(40, rate = 1), rexp(40, rate = 4), rexp(40, rate = 1))
fit <- reliever_em(
  data = x, family = "exp", cpn_max = 3, dm = 10,
  cov_rate = 0.6, method = "BS"
)
selected <- select_by_run(fit, cpn_crit = "sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

---

reliever_generic *Reliever changepoint detection with a custom interval-loss function*

Description

Run a changepoint search with a user-supplied interval loss. This is the primary fitting interface for custom models. The supplied `reg_fun` fits a model using observations `l:r` and returns the loss of every observation in `l_end:r_end`. For a built-in loss family, use `reliever()` instead. Model-specific `reliever_*()` functions expose the built-in implementations with complete formal argument lists.

Usage

```
reliever_generic(
  data,
  reg_fun,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  dc_grid = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  ...
)
```

Arguments

`data`	A vector, matrix, data frame, or model-specific data object with observations in rows. The object must be accepted by `reg_fun`.
`reg_fun`	A function following the interval-loss contract below. It fits on `l:r` and returns observation-level losses on `l_end:r_end`.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`	`NULL` or a reusable cache-profile object returned by a previous call made with `detail = TRUE`. Reuse is valid only when the data, `reg_fun`, its arguments, search grid, `cov_rate`, and `cache_backend` are unchanged.
`run_cpd_ids`	`NULL` or an integer vector of identifiers from the `loss_output_id` column of the virtual metadata table, selecting which output losses to search for changepoints. The default `NULL` fits changepoints for all outputs.
`...`	Additional arguments passed unchanged to `reg_fun`.

Value

A list of class `reliever_result`. Its main components are:

`summary` A data frame containing any explicitly requested fit-time selection; empty by default for a K-indexed path.

`cpd_path` A list containing every distinct candidate segmentation, including its K, changepoints, and stored loss.

`run_meta` A data frame containing the identifiers and optional metadata for each loss output returned by `reg_fun`.

`settings` A named list containing the resolved search, grid, cache, and loss arguments.

`timing` A data frame containing model-fit counts and elapsed times by run.

Requested diagnostics and reusable cache objects are included when available. Use `summary()`, `plot()`, and the post-fit selectors to work with the result.

Writing a custom reg_fun

A single `reg_fun` definition supplies its loss-output count, optional metadata, and all interval losses. `reliever_generic()` uses it in two ways:

- Before the search, it calls `reg_fun(data, l = 1L, r = 1L, is_virtual_run = TRUE, ...)`. This call must return either a positive integer giving the number of loss outputs, or a list with `n_loss_outputs` and optional `loss_output_meta`.
- During the search, `reg_fun` receives fitting endpoints, evaluation endpoints, and `save_model = FALSE`. Here `l:r` is the fitting interval and `l_end:r_end` is the interval whose observations must be scored. The result must be a list with a numeric `loss` matrix having one row for every observation in `l_end:r_end` and one column for every declared loss output. For one loss output, `loss` may instead be a numeric vector of that length.

This is the complete minimum contract: a positive output count for the virtual call and correctly sized losses for each fitted interval. All metadata are optional.

A virtual call may additionally return `loss_output_meta`, with one row per output. Its standard columns are:

- `loss_output_id` and `row_type`;
- `hyper_value` and its display label `hyper_name`;
- `default_selection`; and
- `loss_kind`. Built-in labels are `"rss"`, `"negative_log_likelihood"`, and `"crossfit_loss"`. Custom losses may use another non-empty label.

These fields support output filtering, readable summaries, default plotting, and compatible model selection. Omitted output identifiers are generated automatically, unrecognized columns are retained, and missing `loss_kind` leaves scale compatibility to the author. This model-independent contract gives custom `reg_fun` implementations the common result format and public post-fit selectors. The fitted result also retains the function and its resolved arguments for external segment evaluation; `select_holdout()` therefore does not require them to be entered a second time.

References

Qian, C., Wang, G., and Zou, C. (2025). Reliever: Relieving the burden of costly model fits for changepoint detection. *Journal of Machine Learning Research*, 26(203), 1–57.

Qian, C., Wang, G., Wang, Z., and Zou, C. (2024). Changepoint detection in complex models: Cross-fitting is needed. arXiv:2411.07874.

See Also

reliever(), cv.reliever_generic(), and reg_fun_crossfit_template().

Examples

```
set.seed(2026)
n_seg <- 150
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
custom_mean_loss <- function(data, l, r, l_end = l, r_end = r,
                             save_model = FALSE,
                             is_virtual_run = FALSE) {
  if (is_virtual_run) {
    return(1L)
  }
  data <- as.matrix(data)
  mu <- colMeans(data[l:r, , drop = FALSE])
  residual <- sweep(data[l_end:r_end, , drop = FALSE], 2L, mu)
  list(loss = rowMeans(residual^2))
}
res <- reliever_generic(
  data = x, reg_fun = custom_mean_loss,
  cpn_max = 7, dm = 30, cov_rate = 0.8, method = "SN"
)
summary(res) # empty: the generic fit does not infer a selection rule

# Explicit RSS-SIC is available when the custom loss is known to be RSS.
selected_sic <- select_by_run(result = res, cpn_crit = "rss_sic")
selected_sic
stopifnot(identical(selected_sic$K_hat, 2L))
res$cpd_path$candidates
```

`reliever_glm` *Reliever for generalized-linear-model changes*

Description

Fit one generalized linear model in each interval and use the selected family's observation-level deviance residuals as segment losses. The leading `response_ncol` columns of `data` contain the response. Use two response columns for binomial successes and failures.

Usage

```
reliever_glm(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
```

```
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  family,
  intercept = TRUE,
  response_ncol = 1L
)
```

Arguments

`data`	Numeric matrix with the response column or columns first and predictors afterward.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`family`	A GLM family object, family function, or the name of a `stats` family function.
`intercept`	Add an intercept to each interval model.
`response_ncol`	Number of leading response columns; use 2 for binomial successes and failures.

Details

The main built-in entry is `reliever()` with `cpd_family = "glm"`. Use `cv.reliever()` with the same inputs to select K by outer sample-splitting.

Value

A `reliever_result` object.

See Also

`reg_fun_glm()`, `reliever_lm()`, `cv.reliever()`

Examples

```
set.seed(2026)
n <- 120
x <- rnorm(n)
beta <- rep(c(1.5, -1.5, 0.5), each = 40)
trials <- rep(5L, n)
success <- rbinom(n, trials, plogis(beta * x))
fit <- reliever_glm(
  data = cbind(success, trials - success, x),
  family = binomial(), response_ncol = 2,
  cpn_max = 3, dm = 10, cov_rate = 0.6, method = "BS"
)
# A numeric criterion is an additive penalty per changepoint.
selected <- select_by_run(fit, cpn_crit = 20)
selected
stopifnot(identical(selected$K_hat, 2L))
```

`reliever_kde_l2` *Reliever with fixed-kernel KDE L2 loss*

Description

Detect distributional changes with an empirical-grid kernel-density L_2 loss. For every candidate segment, `reg_fun_kde_l2()` estimates the segment KDE and evaluates the corresponding squared L_2 discrepancy on a fixed common grid. With a fixed density `kernel`, this wrapper computes one kernel-evaluation matrix and reuses it throughout the search; that matrix is a computational representation of the KDE loss, not the statistical target itself.

Usage

```
reliever_kde_l2(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  kernel = NULL,
  bandwidth = NULL,
```

```
  kernel_args = list()
)
```

Arguments

`data`	With `kernel = NULL`, a numeric kernel-evaluation matrix with observations in rows and common evaluation locations in columns; an n by n Gram matrix is the ordinary case. A generic fixed feature matrix is also accepted. For the KDE-L2 interpretation, its columns are fixed kernel-density evaluations on a common grid. Otherwise, `data` contains the raw numeric observations used to construct one kernel-evaluation matrix.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`kernel`	Fixed kernel. The default `NULL` means `data` is already a feature or Gram matrix. Built-in choices are: • `"gaussian"` (or `"rbf"`) and `"laplace"`; • `"linear"` and `"polynomial"`; • `"matern32"` and `"matern52"`; and • `"rational_quadratic"`. Gaussian, radial Laplace, Matérn-3/2, and Matérn-5/2 admit a density-kernel interpretation up to an omitted fixed normalizing constant. Rational quadratic does so in dimension p when `alpha > p / 2`. Linear and polynomial kernels are fixed-feature extensions. A custom function must accept `x`, `y`, and the named values in `kernel_args`, plus `bandwidth` when it is supplied, and return a finite `nrow(x)` by `nrow(y)` matrix. Its full-data matrix must be symmetric.
`bandwidth`	One fixed positive bandwidth for a radial named kernel, or an optional value passed to a custom kernel. Use `NULL` for linear and polynomial kernels.
`kernel_args`	Named list of additional fixed settings. Polynomial accepts `degree = 2`, `scale = 1`, and `offset = 1`. Rational quadratic accepts `alpha = 1`. For a custom function, all entries are passed unchanged.

Details

With `kernel = NULL`, `data` is an already computed kernel-evaluation matrix; a generic fixed feature matrix is also accepted as a fixed-feature extension. The kernel and bandwidth remain fixed throughout the search. The main built-in entry is `reliever(X, cpd_family = "kde_l2", kernel = ..., bandwidth = ...)`; this wrapper exposes all fixed-kernel KDE-L2 arguments.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `select_by_run(fit, cpn_crit = "rss_sic")` to select K by minimizing $n\log(RSS/n)/2 + \log(n)K$. The kernel and bandwidth remain fixed.

References

Padilla, O. H. M., Yu, Y., Wang, D., and Rinaldo, A. (2021). Optimal nonparametric multivariate change point detection and localization. *IEEE Transactions on Information Theory*, 68(3), 1922–1944.

See Also

`reliever()`, `reg_fun_kde_l2()`, `reliever_kde_nll_crossfit()`, `select_by_run()`

Examples

```
# Constructing and searching the KDE-L2 Gram matrix takes over five seconds.
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
bandwidth <- sqrt(10)
fit <- reliever_kde_l2(
  data = x, cpn_max = 7, dm = 30, cov_rate = 0.8, method = "SN",
  kernel = "gaussian", bandwidth = bandwidth
)
selected <- select_by_run(result = fit, cpn_crit = "rss_sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

`reliever_kde_nll` *Reliever with KDE negative-log-likelihood solution-path losses*

Description

Detect distribution changes by fitting one isotropic density kernel over a path of bandwidths. Pass the original observations in rows; the wrapper computes one reusable distance matrix before the changepoint search. A precomputed squared-distance matrix is accepted through `input_type = "distance"` and `var_dim`. The function returns one candidate path per bandwidth. The main built-in entry is `reliever(X, cpd_family = "kde_nll")`; this wrapper exposes all KDE-NLL arguments.

Usage

```
reliever_kde_nll(
  data,
  cpn_max = 3,
  dm = 50,
```

```
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  bandwidth_vec = NULL,
  var_dim = NULL,
  kernel = "gaussian",
  kernel_args = list(),
  input_type = c("auto", "data", "distance")
)
```

Arguments

`data`	Numeric observations in rows, or an n by n precomputed squared-distance matrix.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`bandwidth_vec`	Positive KDE bandwidths h. When `NULL`, a scale-adaptive grid is constructed from 20 log-spaced values b satisfying $b/\sqrt{s_D} \in [1/\sqrt{120}, 1]$, where s_D is the median squared distance among observation pairs separated by at most $\lceil\sqrt{n}\rceil$ time points. Using local pairs estimates the within-segment scale without being dominated by large changes. This reduces to approximately $[\sqrt{p/60}, \sqrt{2p}]$ for standardized p-dimensional Gaussian data. Kernel-specific conversions are: • Gaussian: $h = b$. • Radial Laplace: $h = b/\sqrt{p+1}$. • Student with $\nu > 2$: $h = b\sqrt{(\nu-2)/\nu}$. These give every kernel the same marginal variance at a fixed b. Student kernels with `df <= 2` require an explicit bandwidth vector. An explicit vector is always used unchanged.
`var_dim`	Dimension of the original observations. It is inferred for raw data and required for a precomputed squared-distance matrix.

`kernel` Isotropic density kernel: `"gaussian"`, `"laplace"`, or `"student"`. Here Laplace means the radial L_2 density, not a coordinate-wise product or L_1 kernel. Gaussian uses $\exp\{-\|x-y\|^2/(2h^2)\}$; radial Laplace uses $\exp\{-\|x-y\|/h\}$; and Student uses the multivariate Student density with scale matrix $h^2 I$. Other fixed-kernel choices are documented under `reliever_kde_l2()`.

`kernel_args` Named list of kernel-specific settings. Student accepts `df`, a positive degrees-of-freedom value with default 5. Gaussian and Laplace currently accept no additional settings.

`input_type` Input representation. `"data"` treats rows as raw observations and infers `var_dim`; `"distance"` requires a symmetric squared-distance matrix and explicit `var_dim`. `"auto"`, the default, preserves the former distance-matrix call when a distance-like matrix and `var_dim` are supplied. A distance-like matrix without `var_dim` is rejected as ambiguous; other inputs are treated as raw observations. For an unusually distance-like raw square matrix, set `input_type = "data"` explicitly.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `reliever(X, cpd_family = "kde_nll_crossfit")` for data-driven bandwidth selection by cross-fitted loss. With an independent evaluation sample, `select_holdout()` can instead select bandwidth and K. On an ordinary KDE-NLL fit, `select_by_run(fit, cpn_crit = "sic")` selects K separately for every fixed bandwidth; it does not select a bandwidth.

See Also

`reliever()`, `reg_fun_kde_nll_solpath()`, and `select_by_run()`.

Examples

```
set.seed(2026)
n_seg <- 200
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- reliever_kde_nll(
  data = x, cpn_max = 5, dm = 20, cov_rate = 0.6, # just for speed in example
  method = "SN", bandwidth_vec = c(0.5, 1, 2)
)
# Optional SIC selects K separately at every fixed bandwidth.
selected <- select_by_run(result = fit, cpn_crit = "sic")
selected
# At least one candidate bandwidth should recover the true K.
stopifnot(any(selected$K_hat == 2L))
```

`reliever_kde_nll_crossfit` *Reliever with cross-fitted KDE negative-log-likelihood losses*

Description

Detect distribution changes with isotropic KDE losses cross-fitted separately within each interval. Pass the original observations in rows; one bandwidth-independent distance matrix is computed before searching. Alternatively, supply a precomputed squared-distance matrix. Kernel bandwidth is chosen separately within each fitted interval for the `recv` loss path. The default retains that path and the fixed-bandwidth cross-fitted paths without choosing K or one global bandwidth. The main built-in entry is `reliever(X, cpd_family = "kde_nll_crossfit")`; this wrapper exposes all crossfit KDE-NLL arguments.

Usage

```
reliever_kde_nll_crossfit(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  bandwidth_vec = NULL,
  var_dim = NULL,
  kernel = "gaussian",
  kernel_args = list(),
  input_type = c("auto", "data", "distance")
)
```

Arguments

`data`	Numeric observations in rows, or an n by n precomputed squared-distance matrix. See `input_type`.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`nfolds`	An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.
`fold_type`	A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.
`op_size`	A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.
`buffer_lag`	A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.
`fold_stable_const`	A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.
`loss_output_types`	A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.
`bandwidth_vec`	Positive KDE bandwidths h. When `NULL`, a scale-adaptive grid is constructed from 20 log-spaced values b satisfying $b/\sqrt{s_D} \in [1/\sqrt{120}, 1]$, where s_D is the

median squared distance among observation pairs separated by at most $\lceil\sqrt{n}\rceil$ time points. Using local pairs estimates the within-segment scale without being dominated by large changes. This reduces to approximately $[\sqrt{p/60}, \sqrt{2p}]$ for standardized p-dimensional Gaussian data. Kernel-specific conversions are:

- Gaussian: $h = b$.
- Radial Laplace: $h = b/\sqrt{p+1}$.
- Student with $\nu > 2$: $h = b\sqrt{(\nu-2)/\nu}$.

These give every kernel the same marginal variance at a fixed b. Student kernels with `df <= 2` require an explicit bandwidth vector. An explicit vector is always used unchanged.

`var_dim` Dimension of the original observations. It is inferred for raw data and required for a precomputed squared-distance matrix.

`kernel` Isotropic density kernel: `"gaussian"`, `"laplace"`, or `"student"`. Here Laplace means the radial L_2 density, not a coordinate-wise product or L_1 kernel. Gaussian uses $\exp\{-\|x-y\|^2/(2h^2)\}$; radial Laplace uses $\exp\{-\|x-y\|/h\}$; and Student uses the multivariate Student density with scale matrix $h^2 I$. Other fixed-kernel choices are documented under `reliever_kde_l2()`.

`kernel_args` Named list of kernel-specific settings. Student accepts `df`, a positive degrees-of-freedom value with default 5. Gaussian and Laplace currently accept no additional settings.

`input_type` Input representation. `"data"` treats rows as raw observations and infers `var_dim`; `"distance"` requires a symmetric squared-distance matrix and explicit `var_dim`. `"auto"`, the default, preserves the former distance-matrix call when a distance-like matrix and `var_dim` are supplied. A distance-like matrix without `var_dim` is rejected as ambiguous; other inputs are treated as raw observations. For an unusually distance-like raw square matrix, set `input_type = "data"` explicitly.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

- Interval-adaptive ReCV uses `select_by_run()` with run type `"recv"` and criterion `"loss"`.
- Homogeneous ReCV jointly selects one bandwidth and K from stored fixed-bandwidth rows.

Apply `select_across_runs()` for the homogeneous workflow.

Its run type is `"crossfit_homo_hyper"`; request both crossfit outputs when fitting. Replace `"loss"` with `"sic"` in either selector to minimize $ReCV\ loss + \log(n)K$.

To check the automatic bandwidth range at a chosen K, use a hyperparameter plot with run type `"crossfit_homo_hyper"`.

See Also

`reliever()`, `reg_fun_kde_nll_crossfit()`, and `select_by_run()`.

Examples

```
# This cross-fitted KDE example takes more than five seconds.
set.seed(2026)
n_seg <- 150
```

```
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- reliever_kde_nll_crossfit(
  data = x, cpn_max = 5, dm = 15, cov_rate = 0.6, # for speed in example
  kernel = "student", kernel_args = list(df = 5),
  nfolds = 2,
  loss_output_types = c("recv", "crossfit_homo_hyper"),
  method = "SN"
)
selected <- select_by_run(
  result = fit, run_type = "recv", cpn_crit = "sic"
)
selected
stopifnot(identical(selected$K_hat, 2L))
selected_fixed <- select_across_runs(
  result = fit, run_type = "crossfit_homo_hyper", cpn_crit = "sic"
)
selected_fixed
stopifnot(identical(selected_fixed$K_hat, 2L))
plot(fit, run_type = "recv")
plot(
  fit, x_axis = "hyperparameter", K = selected$K_hat[[1L]],
  run_type = "crossfit_homo_hyper", cpn_crit = "sic", log = "x"
)
```

`reliever_lasso` *Reliever with lasso solution-path losses*

Description

Run Reliever with glmnet lasso losses over a common normalized lambda path. A stored value `lam_set` is sample-size independent: an interval with m rows passes $lam_set/\sqrt{m}$ to `glmnet`. Use this when the response is in the first column of `data` and the remaining columns are predictors. For each lambda, the function computes candidate segmentations with different numbers of change-points. The main built-in entry is `reliever(X, y, cpd_family = "lasso")`. This wrapper exposes all lasso arguments and accepts the equivalent response-first `data` matrix. If `lam_set = NULL`, it asks a full-data glmnet fit for a path controlled by `nlambda`, rescales it to Reliever's convention, and extends both ends by two values. This constructs a candidate range; it does not select lambda. Supply `lam_set` to use a fixed user-defined path instead. Built-in lasso fits use `intercept = FALSE` and `standardize = FALSE`; put predictors on their intended scale before analysis.

Usage

```
reliever_lasso(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
```

```
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  lam_set = NULL,
  nlambda = 30L,
  family = "gaussian",
  thresh = 1e-07
)
```

Arguments

`data`	Numeric matrix with the response in the first column and at least two predictors in the remaining columns, as required by `glmnet`.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`lam_set`	Optional finite positive normalized lambda values. The default `NULL` constructs one data-adaptive path and reuses it throughout the changepoint search. The low-level lasso-path help page defines the scale.
`nlambda`	Integer scalar of at least 2 passed to the full-data `glmnet` fit that constructs the automatic path when `lam_set = NULL`. The default is 30. It is ignored when an explicit `lam_set` is supplied.
`family`	Response family: `"gaussian"`, `"binomial"`, or `"poisson"`. Gaussian returns squared prediction loss; binomial and Poisson return negative log-likelihood loss. Binomial responses must be 0 or 1; Poisson responses may contain any finite non-negative values.
`thresh`	Convergence threshold passed to `glmnet::glmnet()`.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `cv.reliever(..., cpd_family = "lasso")` to select lambda and K by CPSS-style outer cross-validation. For Gaussian loss, `select_by_run(fit, cpn_crit = "rss_sic")` selects K separately for each fitted lambda path; binomial and Poisson losses use `"sic"` instead.

See Also

`reliever()`, `cv.reliever()`, and `reg_fun_lasso_solpath()`.

Examples

```
# This high-dimensional lasso example takes more than five seconds.
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
data <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data

# Fit the in-sample lambda paths; no K or model setting is selected.
fit_in <- reliever_lasso(
  data = data, cpn_max = 7, dm = 30, cov_rate = 0.8, method = "SN"
)
summary(fit_in)

# For Gaussian loss, RSS-SIC can select K separately at every lambda.
selected <- select_by_run(result = fit_in, cpn_crit = "rss_sic")
selected
# At least one candidate lambda should recover the true K.
stopifnot(any(selected$K_hat == 2L))
```

---

reliever_lasso_crossfit

Reliever with cross-fitted lasso losses

Description

Run Reliever with lasso losses built by interval-level cross-fitting. The result contains a cross-fitted path for every normalized lambda setting held fixed across all intervals, plus an interval-adaptive recycled-CV (`recv`) path in which that setting is chosen separately within every fitted interval. A fold with m training rows passes $lam_set/\sqrt{m}$ to `glmnet`. The default `cpn_crit = "none"` fits and retains these paths without choosing K or one global normalized lambda setting. If `lam_set = NULL`, a data-adaptive full-data glmnet path is converted to this normalized scale once and reused for every interval. Built-in lasso fits use `intercept = FALSE` and `standardize = FALSE`; put predictors on their intended scale before analysis. The main built-in entry is `reliever(X, y, cpd_family = "lasso_crossfit")`. This wrapper exposes all crossfit-lasso arguments and accepts response-first `data`.

Usage

```
reliever_lasso_crossfit(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  lam_set = NULL,
  nlambda = 30L,
  family = "gaussian",
  thresh = 1e-07
)
```

Arguments

`data`	Numeric matrix with the response in the first column and at least two predictors in the remaining columns, as required by `glmnet`.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`nfolds`	An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.

`fold_type` A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.

`loss_output_types` A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`lam_set` Optional finite positive normalized lambda values. The default `NULL` constructs one data-adaptive path and reuses it throughout the changepoint search. The low-level crossfit-lasso help page defines the scale.

`nlambda` Integer scalar of at least 2 passed to the full-data `glmnet` fit that constructs the automatic path when `lam_set = NULL`. The default is 30. It is ignored when an explicit `lam_set` is supplied.

`family` Response family: `"gaussian"`, `"binomial"`, or `"poisson"`. Gaussian returns squared prediction loss; binomial and Poisson return negative log-likelihood loss. Binomial responses must be 0 or 1; Poisson responses may contain any finite non-negative values.

`thresh` Convergence threshold passed to `glmnet::glmnet()`.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

- Interval-adaptive ReCV uses `select_by_run()` with run type `"recv"` and criterion `"loss"`.

- Joint selection of one normalized lambda and K applies `select_across_runs()` to run type `"crossfit_homo_hyper"`. Request both crossfit output types when fitting.

Replace `"loss"` with `"sic"` in either selector to minimize $ReCV\ loss + \log(n)K$. Inspect fixed-setting losses with a hyperparameter plot at the chosen K.

See Also

`reliever()`, `reg_fun_lasso_crossfit()`, and `select_by_run()`.

Examples

```
# This cross-fitted lasso example takes more than five seconds.
set.seed(2026)
n <- 450
p <- 20
tau <- c(150, 300)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
data <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
fit <- reliever_lasso_crossfit(
  data = data, cpn_max = 5, dm = 15, cov_rate = 0.8,
  nfolds = 2,
  loss_output_types = c("recv", "crossfit_homo_hyper"),
  method = "SN"
)
selected <- select_by_run(
  result = fit, run_type = "recv", cpn_crit = "loss"
)
selected
stopifnot(identical(selected$K_hat, 2L))

# Homogeneous-hyperparameter ReCV: select one normalized setting and K.
selected_fixed <- select_across_runs(
  result = fit, run_type = "crossfit_homo_hyper", cpn_crit = "loss"
)
selected_fixed
stopifnot(identical(selected_fixed$K_hat, 2L))
plot(x = fit, run_type = "recv")
plot(
  x = fit, x_axis = "hyperparameter", K = 2,
  run_type = "crossfit_homo_hyper", cpn_crit = "loss", log = "x"
)
```

`reliever_lm` *Reliever for linear-model changes*

Description

Fit one ordinary least-squares model in each interval and use observation-level squared residuals as the segment loss. The first column of `data` is the response and the remaining columns are predictors.

Usage

```
reliever_lm(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  intercept = TRUE
)
```

Arguments

`data`	Numeric matrix with the response first and predictors afterward.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`intercept`	Add an intercept to each interval model.

Details

The main built-in entry is `reliever()` with `cpd_family = "lm"`. Use `cv.reliever()` with the same inputs to select K by outer sample-splitting.

Value

A `reliever_result` object.

See Also

`reg_fun_lm()`, `reliever_glm()`, `cv.reliever()`

Examples

```
set.seed(2026)
n <- 120
x <- rnorm(n)
beta <- rep(c(1, -1, 0.5), each = 40)
y <- beta * x + rnorm(n, sd = 0.4)
fit <- reliever_lm(
  data = cbind(y, x), cpn_max = 3, dm = 10,
  cov_rate = 0.6, method = "BS"
)
selected <- select_by_run(fit, cpn_crit = "sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

---

reliever_mean *Reliever for mean-change loss*

Description

A focused wrapper for mean-change detection. Ungridded searches use the package's native C++ mean-loss engine. For a gridded search, prefer `dc_grid_size`; the same mean loss then runs through the generic gridded backend and returns changepoints on the original time scale. The main built-in entry is `reliever(X)`; this wrapper exposes all mean-specific arguments.

Usage

```
reliever_mean(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  dc_grid = NULL,
  ratio = 0.9
)
```

Arguments

`data`	A numeric matrix or vector with observations in rows.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.

`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`
: Common penalty and search controls; see `reliever()`.

`detail`, `cache_backend`, `owner_key`, `echo`
: Common output and cache controls; see `reliever()`.

`dc_grid_size`, `dc_grid`
: Common candidate-grid controls; see `reliever()`.

`ratio`
: Computational tuning parameter in `(0, 1]`. When the next fitted interval differs from the previous one in no more than `ratio` times the new interval length, the C++ engine updates its mean and sum of squares incrementally; otherwise it recomputes them from the new interval. This affects speed, not the fitted criterion, and is ignored for a gridded search.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `cv.reliever()` for CPSS-style outer sample-splitting selection of K. As an information-criterion alternative, `select_by_run(fit, cpn_crit = "rss_sic")` minimizes $n\log(RSS/n)/2+\log(n)K$ on an existing fit.

See Also

`reliever()`, `cv.reliever()`, `reliever_mean_crossfit()`, `reg_fun_mean()`, `select_by_run()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
res <- reliever_mean(data = x, cpn_max = 7, dm = 30, cov_rate = 0.8,
                     method = "SN", echo = FALSE)
summary(res) # empty: fitting did not choose K

# Optional information-criterion selection from the stored path:
selected_sic <- select_by_run(result = res, cpn_crit = "rss_sic")
selected_sic
stopifnot(identical(selected_sic$K_hat, 2L))
res$cpd_path$candidates
```

---

reliever_meanvar *Reliever for simultaneous mean and covariance changes*

Description

Fit a single changepoint path using multivariate Gaussian negative twice log-likelihood, estimating both the mean and maximum-likelihood covariance separately in every fitted interval.

Usage

```
reliever_meanvar(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.

Details

The main built-in entry is `reliever()` with `cpd_family = "meanvar"`. Use `cv.reliever()` with the same family to select K by outer sample-splitting.

Value

A `reliever_result` object.

See Also

`reg_fun_meanvar()`, `reliever_var()`, `cv.reliever()`

Examples

```
set.seed(2026)
x <- c(
  rnorm(40, mean = 0, sd = 0.5),
  rnorm(40, mean = 2, sd = 1.5),
  rnorm(40, mean = -1, sd = 0.7)
)
fit <- reliever_meanvar(
  data = x, cpn_max = 3, dm = 10, cov_rate = 0.6, method = "BS"
)
# A numeric criterion is an additive penalty per changepoint.
selected <- select_by_run(fit, cpn_crit = 20)
selected
stopifnot(identical(selected$K_hat, 2L))
```

`reliever_mean_crossfit` *Reliever with cross-fitted mean-square losses*

Description

Run Reliever with mean-square losses built by interval-level recycled cross-validation (ReCV). The main built-in entry is `reliever(X, cpd_family = "mean_crossfit")`; this wrapper exposes all mean-crossfit arguments.

Usage

```
reliever_mean_crossfit(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
```

```
  fold_stable_const = 1,
  loss_output_types = "recv"
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`nfolds`	An integer scalar giving the number of folds constructed within each fitted interval. The interval length need not be divisible by `nfolds`. It must be an integer of at least 2 and cannot exceed the number of rows in the fitted interval.
`fold_type`	A character scalar selecting the fold construction. `"op"` assigns interlaced, order-preserving fold labels; `"blk"` divides the current fitting interval into contiguous blocks; `"stable_blk"` uses contiguous blocks anchored to the full time axis so that nearby intervals have more stable fold labels. The default is `"op"`.
`op_size`	A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.
`buffer_lag`	A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.
`fold_stable_const`	A numeric scalar of at least 1 giving, for `fold_type = "stable_blk"`, the geometric scale factor used to choose a globally anchored block width. The block width is based on the largest value in $n, n/c, n/c^2, \ldots$ that does not exceed the fitted interval length, where c is `fold_stable_const`, and is then divided among `nfolds`. Thus values larger than 1 keep nearby interval lengths on the same global fold grid; the default 1 scales the width directly with the current interval.
`loss_output_types`	A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `select_by_run(fit, run_type = "recv", cpn_crit = "loss")` for unpenalized ReCV selection of K. Replace `"loss"` with `"sic"` in that call to minimize $ReCV\ loss + \log(n)K$.

See Also

`reliever()`, `reg_fun_mean_crossfit()`, `select_by_run()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- reliever_mean_crossfit(
  data = x, cpn_max = 7, dm = 30, cov_rate = 0.8,
  nfolds = 2, method = "SN"
)
selected <- select_by_run(
  result = fit, run_type = "recv", cpn_crit = "loss"
)
selected
stopifnot(identical(selected$K_hat, 2L))

# A conservative alternative is:
selected_sic <- select_by_run(
  result = fit, run_type = "recv", cpn_crit = "sic"
)
selected_sic
stopifnot(identical(selected_sic$K_hat, 2L))
```

---

reliever_mlp_crossfit *Reliever with cross-fitted MLP-classifier losses*

Description

Detect distribution changes by training a small multilayer perceptron (MLP) to distinguish observations inside a candidate interval from those outside it. Predicted class probabilities are converted to interval-level ReCV losses. This uses the optional `nnet` package. The main built-in entry is `reliever(X, cpd_family = "mlp_crossfit")`; this wrapper exposes all MLP-specific arguments.

Usage

```
reliever_mlp_crossfit(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  hyper_set = data.frame(size = 8),
  nnet_args = list()
)
```

Arguments

`data`	Numeric matrix with observations in rows and classifier features in columns.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`nfolds`	An integer scalar of at least 2 giving the number of global `"op"` folds. The fitted interval must contain at least `nfolds` observations and span at least two fold labels.
`fold_type`	A character scalar selecting fold construction. Classifier cross-fitting fits against the full data and supports only the globally anchored, interlaced `"op"` construction.

`op_size` A positive integer scalar giving, for `fold_type = "op"`, the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.

`buffer_lag` A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at `floor(n_core / (2 * nfolds))` so that every fold remains trainable; it can therefore be 0 on short intervals.

`fold_stable_const` Unused for classifier cross-fitting, which supports only `"op"` folds. Retained for argument compatibility.

`loss_output_types` A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default `"recv"` returns only the interval-adaptive cross-fitted loss. Use `c("recv", "incv")` to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include `"crossfit_homo_hyper"` to additionally return one cross-fitted output per fixed hyperparameter for subsequent `select_across_runs()` selection. `"recv"` is required; an input that omits it produces an error. Returned columns always follow the order `recv`, `incv` when requested, then `crossfit_homo_hyper` in `hyper_set` order.

`hyper_set` A data frame, matrix, or list of candidate MLP settings. Supply a data frame or matrix with one row per setting and uniquely named columns, or a list whose elements are named argument lists.

`nnet_args` A named list of fixed `nnet::nnet()` arguments shared by every candidate in `hyper_set`. Observation `weights` are rejected because the built-in probability conversion requires the empirical training class proportion. The wrapper manages `x` and `y`; do not include them in either argument list. Weighted models require the generic classifier template with recalibrated probabilities. Supply each argument in only one of `hyper_set` and `nnet_args`.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

- Interval-adaptive ReCV uses `select_by_run()` with run type `"recv"` and criterion `"loss"`.
- Joint selection of one classifier setting and K applies `select_across_runs()` to run type `"crossfit_homo_hyper"`.

Request those paths through `loss_output_types` when fitting. Replace `"loss"` with `"sic"` in either call to add a $\log(n)K$ penalty.

See Also

`reliever()`, `reg_fun_mlp_crossfit()`, and `select_by_run()`.

Examples

```
# This optional-model example takes more than five seconds.
if (requireNamespace("nnet", quietly = TRUE)) {
  set.seed(2026)
  n_seg <- 300
  x <- rbind(
    matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
  )
  set.seed(2026)
  fit <- suppressWarnings(reliever_mlp_crossfit(
    data = x, cpn_max = 7, dm = 30, cov_rate = 0.8, nfolds = 2,
    hyper_set = data.frame(size = 2, maxit = 30),
    method = "SN"
  ))
  # Numeric criteria supply an additive penalty per changepoint.
  selected <- select_by_run(
    result = fit, run_type = "recv", cpn_crit = 20
  )
  selected
  stopifnot(identical(selected$K_hat, 2L))
}
```

`reliever_nmcd` *Reliever with univariate nonparametric CDF loss*

Description

Detect arbitrary changes in a univariate distribution by aggregating empirical-CDF log losses. Unlike mean-change detection, this can respond to changes in location, scale, or distributional shape. The main built-in entry is `reliever(x, cpd_family = "nmcd")`; this wrapper exposes all empirical-CDF arguments.

Usage

```
reliever_nmcd(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
```

```
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  w_trunc = 0,
  sort_X = NULL
)
```

Arguments

`data`	Numeric vector or one-column matrix.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`w_trunc`	Fraction removed from each tail of the ordered full-sample CDF cutpoints. If the reference vector has length m, `floor(w_trunc * m)` cutpoints are omitted from both the lower and upper tails. The default 0 keeps every cutpoint used by the criterion.
`sort_X`	Optional sorted reference vector that defines the empirical-CDF cutpoints. The default sorts the training observations in `data`; externally appended evaluation rows are excluded automatically. Model wrappers may resolve this reference once and reuse it.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

Use `select_by_run(fit, cpn_crit = "sic")` to select K by minimizing $CDF\ loss + \log(n)K$. A non-negative numeric criterion supplies another additive penalty per changepoint.

References

Zou, C., Yin, G., Feng, L., and Wang, Z. (2014). Nonparametric maximum likelihood approach to multiple change-point problems. *The Annals of Statistics*, 42(3), 970–1002. DOI: 10.1214/14-AOS1210.

See Also

`reliever()`, `reg_fun_nmcd()`, `select_by_run()`

Examples

```
set.seed(2026)
n_seg <- 300
x <- c(
  rnorm(n_seg, mean = 0, sd = 0.2),
  rnorm(n_seg, mean = 4, sd = 0.2),
  rnorm(n_seg, mean = -4, sd = 0.2)
)
fit <- reliever_nmcd(
  data = x, cpn_max = 7, dm = 30, cov_rate = 0.8, method = "SN"
)
selected <- select_by_run(result = fit, cpn_crit = "sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

---

reliever_ranger_crossfit

Reliever with cross-fitted ranger-classifier losses

Description

Detect distribution changes by training ranger classifiers to distinguish observations inside a candidate interval from those outside it. Predicted class probabilities are converted to interval-level ReCV losses. This requires the optional `ranger` package. The main built-in entry is `reliever()` with `cpd_family = "ranger_crossfit"`; this wrapper exposes all ranger-specific arguments.

Usage

```
reliever_ranger_crossfit(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  nfolds = 5,
  fold_type = "op",
  op_size = 1,
```

```
  buffer_lag = 0,
  fold_stable_const = 1,
  loss_output_types = "recv",
  hyper_set = list(list()),
  ranger_args = list()
)
```

Arguments

data	Numeric matrix or data frame with observations in rows and classifier features in columns.
cpn_max, dm, cov_rate, method, cpn_crit	Common path controls; see reliever().
pen_val, prune_value, M, wbs_seed, wbs_stop_crit	Common penalty and search controls; see reliever().
detail, cache_backend, owner_key, echo	Common output and cache controls; see reliever().
dc_grid_size, dc_grid	Common candidate-grid controls; see reliever().
cache_profile, run_cpd_ids	Advanced reusable-cache and loss-output controls; see reliever_generic().
nfolds	An integer scalar of at least 2 giving the number of global "op" folds. The fitted interval must contain at least nfolds observations and span at least two fold labels.
fold_type	A character scalar selecting fold construction. Classifier cross-fitting fits against the full data and supports only the globally anchored, interlaced "op" construction.
op_size	A positive integer scalar giving, for fold_type = "op", the number of consecutive observations given the same fold label before cycling to the next fold. It must be a positive integer and small enough that the fitted interval spans at least two fold labels.
buffer_lag	A non-negative integer scalar giving the number of time indices excluded on each side of held-out observations when fitting a fold model. Use a positive value to reduce leakage under local temporal dependence; 0 applies no buffer. Within each fitted interval, the effective lag is capped at floor(n_core / (2 * nfolds)) so that every fold remains trainable; it can therefore be 0 on short intervals.
fold_stable_const	Unused for classifier cross-fitting, which supports only "op" folds. Retained for argument compatibility.
loss_output_types	A character vector specifying the crossfit outputs. It is treated as an unordered set: input order does not affect either the calculation or output-column order. The default "recv" returns only the interval-adaptive cross-fitted loss. Use c("recv", "incv") to additionally return the matching in-sample loss chosen by the same interval-specific hyperparameter. Include "crossfit_homo_hyper" to additionally return one cross-fitted output per fixed hyperparameter for subsequent select_across_runs() selection. "recv" is required; an input that omits it produces an error. Returned columns always follow the order recv, incv when requested, then crossfit_homo_hyper in hyper_set order.

`hyper_set` A data frame, matrix, or list of candidate ranger settings. Supply a data frame or matrix with one row per setting and uniquely named columns, or a list whose elements are named argument lists.

`ranger_args` A named list of fixed `ranger::ranger()` arguments shared by every candidate in `hyper_set`. The built-in conversion rejects class weights, case weights, and class-specific sampling because it requires probabilities under the empirical training class proportion. The wrapper manages `formula`, `data`, `write.forest`, and `probability`; do not include them in either argument list. Other controls, including `num.threads`, may be supplied. Supply each argument in only one of `hyper_set` and `ranger_args`. Weighted classifiers require the generic classifier template with recalibrated probabilities.

Value

A `reliever_result` object with the common structure documented in `reliever()`.

Selection

- Interval-adaptive ReCV uses `select_by_run()` with run type `"recv"` and criterion `"loss"`.
- Joint selection of one classifier setting and K applies `select_across_runs()` to run type `"crossfit_homo_hyper"`.

Request those paths through `loss_output_types` when fitting. Replace `"loss"` with `"sic"` in either call to add a $\log(n)K$ penalty.

References

Londschien, M., Bühlmann, P., and Kovács, S. (2023). Random forests for change point detection. *Journal of Machine Learning Research*, 24(216), 1–45.

See Also

`reliever()`, `reg_fun_ranger_crossfit()`, and `select_by_run()`.

Examples

```
# This optional-model example takes more than five seconds.
if (requireNamespace("ranger", quietly = TRUE)) {
  set.seed(2026)
  n_seg <- 300
  x <- rbind(
    matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
    matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
  )
  fit <- suppressWarnings(reliever_ranger_crossfit(
    data = x, cpn_max = 7, dm = 30, cov_rate = 0.8, nfolds = 2,
    hyper_set = data.frame(num.trees = 10, min.node.size = 5),
    ranger_args = list(seed = 2026), method = "SN"
  ))
  selected <- select_by_run(
    result = fit, run_type = "recv", cpn_crit = "loss"
  )
  selected
  stopifnot(identical(selected$K_hat, 2L))
```

```
}
```

`reliever_var` *Reliever for covariance changes*

Description

Fit a single changepoint path using multivariate Gaussian negative twice log-likelihood, with one common mean and interval-specific covariance matrices. If `mu = NULL`, the common mean is estimated once from the complete input and then held fixed in every interval.

Usage

```
reliever_var(
  data,
  cpn_max = 3,
  dm = 50,
  cov_rate = 0.8,
  method = "SN",
  cpn_crit = "none",
  pen_val = 1,
  prune_value = 0,
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  detail = FALSE,
  cache_backend = "by_loss_block",
  owner_key = TRUE,
  echo = FALSE,
  dc_grid_size = NULL,
  cache_profile = NULL,
  run_cpd_ids = NULL,
  dc_grid = NULL,
  mu = NULL
)
```

Arguments

`data`	Numeric vector or matrix with observations in rows.
`cpn_max`, `dm`, `cov_rate`, `method`, `cpn_crit`	Common path controls; see `reliever()`.
`pen_val`, `prune_value`, `M`, `wbs_seed`, `wbs_stop_crit`	Common penalty and search controls; see `reliever()`.
`detail`, `cache_backend`, `owner_key`, `echo`	Common output and cache controls; see `reliever()`.
`dc_grid_size`, `dc_grid`	Common candidate-grid controls; see `reliever()`.
`cache_profile`, `run_cpd_ids`	Advanced reusable-cache and loss-output controls; see `reliever_generic()`.
`mu`	Optional fixed common mean. A scalar is applied to every column.

Details

The main built-in entry is `reliever()` with `cpd_family = "var"`. Use `cv.reliever()` with the same family to select K by outer sample-splitting.

Value

A `reliever_result` object.

See Also

`reg_fun_var()`, `reliever_meanvar()`, `cv.reliever()`

Examples

```
set.seed(2026)
x <- c(rnorm(40, sd = 0.5), rnorm(40, sd = 2), rnorm(40, sd = 0.5))
fit <- reliever_var(
  data = x, cpn_max = 3, dm = 10, cov_rate = 0.6, method = "BS"
)
selected <- select_by_run(fit, cpn_crit = "sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

---

select_across_runs *Select one model across comparable fitted loss paths*

Description

Select one candidate jointly across comparable stored loss paths. Homogeneous-hyperparameter ReCV (`recv_homo_hyper`) uses:

```
select_across_runs(
  result = fit, run_type = "crossfit_homo_hyper", cpn_crit = "loss"
)
```

This jointly selects one fixed hyperparameter, K, and the changepoints. Compared paths must use the same loss definition and scale.

Usage

```
select_across_runs(result, run_ids = NULL, run_type = NULL, cpn_crit, n = NULL)
```

Arguments

`result`	A `reliever_result` returned by `reliever()`, `reliever_generic()`, or a focused model-specific wrapper.
`run_ids`	Optional exact integer identifiers from `result$run_meta`. Use these for direct path-level control. Supply either this argument or `run_type` so that unrelated losses, such as in-sample and cross-fitted losses, are not compared accidentally.
`run_type`	Optional row types to compare. Supply either this argument or `run_ids`.

- `"recv"` identifies interval-adaptive ReCV paths.

- `"crossfit_homo_hyper"`: fixed-hyperparameter cross-fitted paths.
- Other types remain available but trigger a comparability warning. A type containing only one run also suggests `select_by_run()` when only K is being selected.

For a custom `reg_fun`, advanced users should prefer explicit `run_ids` for deliberately comparable paths.

`cpn_crit` Required rule used to compare K and runs. It accepts the rules documented in `select_by_run()`; `"loss"` is unpenalized. Supply this argument explicitly so the statistical choice is visible.

`n` Sample size used in HQC/SIC penalties and the RSS transformation. The default reads the original observation count from `result$settings`; a compressed candidate grid does not change it. Supply `n` only for a manually constructed result that lacks this information.

Value

One row containing the jointly selected `K_hat`, `cpd_hat`, minimized `score`, and available model metadata. Internal path identifiers are shown by `print(x, details = TRUE)`; see `select_by_run()` for the common column definitions.

Guidance warnings

Selection continues after each guidance warning:

- Unusual run types signal `"reliever_across_run_comparability_warning"`.
- A type resolving to one run signals `"reliever_single_run_selection_warning"`.
- Different declared loss kinds signal `"reliever_loss_kind_comparability_warning"`.

Explicit `run_ids` bypass the run-type guidance warnings.

See Also

`summary.reliever_result()`, `select_by_run()`, `select_holdout()`

Examples

```
# Fitting the cross-fitted lasso paths takes more than five seconds.
set.seed(2026)
n <- 450
p <- 20
tau <- c(150, 300)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
dat <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
fit <- reliever(
  X = dat[, -1, drop = FALSE], y = dat[, 1],
  cpd_family = "lasso_crossfit",
  cpn_max = 5, dm = 15, cov_rate = 0.8, method = "SN",
  nfolds = 2,
  loss_output_types = c("recv", "crossfit_homo_hyper")
)
# crossfit_homo_hyper means cross-fitted loss with one interval-independent
# hyperparameter value.
selected <- select_across_runs(
```

```
  result = fit, run_type = "crossfit_homo_hyper", cpn_crit = "loss"
)
selected
stopifnot(identical(selected$K_hat, 2L))
```

`select_by_run` *Select K separately within fitted loss paths*

Description

A run is one stored loss path. This function selects K separately in every requested run and returns one row per run. Use `run_type` or `run_ids` to choose the paths. Use `select_across_runs()` when one model setting and K should be selected jointly across comparable paths. If a WBS-family fit supplied `wbs_stop_crit`, only the segmentations mapped from those thresholds and the no-change baseline are compared. Omit `wbs_stop_crit` when selection should traverse the full K path.

Usage

```
select_by_run(result, run_ids = NULL, run_type = NULL, cpn_crit, n = NULL)
```

Arguments

`result`	A `reliever_result` returned by `reliever()`, `reliever_generic()`, or a focused model-specific wrapper.
`run_ids`	Optional exact integer identifiers from `result$run_meta`. Use these when individual paths must be controlled directly. If neither `run_ids` nor `run_type` is supplied, K is selected separately for every run.
`run_type`	Optional run-type labels, such as `"recv"` or `"crossfit_homo_hyper"`. These are matched against the `row_type` metadata field. This is the preferred way to select a statistically defined group of stored paths. It filters the runs before candidate values of K are compared; it is not itself a selection criterion. Supply only one of `run_ids` and `run_type`.
`cpn_crit`	Required rule used to compare candidates with different K. Use `"loss"` to choose the smallest stored loss with no K penalty. Use `"aic"`, `"hqc"`, or `"sic"` to minimize $loss + \gamma K$, where γ is 2, $2\log\log(n)$, or $\log(n)$. These additive rules may also be applied directly to RSS. The alternatives `"rss_aic"`, `"rss_hqc"`, and `"rss_sic"` instead minimize $n\log(RSS/n)/2 + \gamma K$; this log-RSS form may be preferable when residual variance is unknown or potentially heterogeneous. A non-negative number supplies γ directly in $loss + \gamma K$. See `reliever()` for the same rules in the fitting API.
`n`	Sample size used in HQC/SIC penalties and the RSS transformation. The default reads the original observation count from `result$settings`; a compressed candidate grid does not change it. Supply `n` only for a manually constructed result that lacks this information.

Value

One row per requested run, containing `K_hat`, `cpd_hat`, the minimized `score`, and available model metadata. Internal `run_id` and `candidate_id` columns link back to the fitted paths and are shown by `print(x, details = TRUE)`. A requested run with no finite candidate score is retained, with `NA` selection fields.

See Also

summary.reliever_result(), select_across_runs(), select_holdout()

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- reliever(X = x, cpn_max = 7, dm = 30, cov_rate = 0.8, method = "SN")
selected <- select_by_run(result = fit, cpn_crit = "rss_sic")
selected
stopifnot(identical(selected$K_hat, 2L))
```

---

select_holdout *Select a fitted candidate path by hold-out loss*

Description

Refit every requested candidate segmentation on data, score it on independent eval_data, and return the candidate with the smallest hold-out loss.

Usage

```
select_holdout(
  result,
  data,
  eval_data,
  eval_index = NULL,
  y = NULL,
  eval_y = NULL,
  K = NULL,
  run_ids = NULL,
  reg_fun = NULL,
  data_stack_fun = NULL,
  ...,
  run_type = NULL
)
```

Arguments

result	A fitted reliever_result object.
data	Training observations. For a built-in lasso fit, this may be either the predictor matrix used in reliever(X, y, ...) when y is supplied here, or a response-first matrix when y is omitted. Built-in kernel fits accept the same raw or precomputed representation described in evaluate_reliever_segments().

eval_data	Independent observations used to compare candidate segmentations. For a built-in lasso fit, use an evaluation predictor matrix with eval_y, or a response-first matrix without eval_y. Kernel inputs and their rectangular precomputed forms are described in evaluate_reliever_segments().
eval_index	Time index for each row of eval_data. For one evaluation observation at every original time point, the default NULL uses seq_len(n), where n is the number of training observations. Supply explicit indices for sparse, repeated, or otherwise non-one-to-one evaluation observations.
y, eval_y	Optional training and evaluation responses for a built-in lasso or lasso-crossfit fit. Supply both with predictor-only data/eval_data; omit both with response-first matrices. Their values must satisfy the response family stored in the fit.
K	Optional fixed changepoint number. When omitted, all requested K candidates are compared; fixed-setting paths can also select their model setting.
run_ids	Optional identifiers from result$run_meta. The default compares every fitted run; supply a subset to compare only selected model settings. Supply only one of run_ids and run_type.
reg_fun	Optional compatible loss-function override. The default reuses the function retained in result; see evaluate_reliever_segments().
data_stack_fun	Function used to combine data and eval_data; see the segment-evaluation help page.
...	Named loss-function arguments for a manually constructed result, additional evaluation-only arguments, or explicit overrides of retained arguments. An override must preserve the selected loss-output identifiers and hyperparameter path.
run_type	Optional character values matched against resultrun_metarow_type, such as "recv" or "crossfit_homo_hyper". This is the preferred way to restrict hold-out comparison to a statistically defined kind of stored loss path. Supply only one of run_ids and run_type.

Details

With K = NULL, the function selects among the requested K/path candidates; fixed-setting paths jointly select K and that model setting. For a crossfit result, use run_type = "recv" for interval-adaptive tuning or run_type = "crossfit_homo_hyper" for one homogeneous setting. Supplying K fixes the changepoint number.

Use run_ids or run_type to restrict the paths. Compared paths must use the same observation-level scoring rule, units, and normalization. Built-in external scoring is available for ordinary losses, lasso-crossfit, and KDE-NLL-crossfit. Other crossfit templates require a model-specific external scorer.

If a WBS-family fit supplied wbs_stop_crit, only the segmentations mapped from those thresholds and the no-change baseline are compared.

Value

One row containing the selected K_hat, cpd_hat, hold-out loss in score, and hyper_value when available. If the fit contains several statistically distinct loss-row types, row_type identifies the selected source. Internal run_id and candidate_id columns remain available for path tracing but are hidden by the default print method; use print(x, details = TRUE) to show them. Use evaluate_reliever_segments() when losses for every candidate are needed.

See Also

reliever(), evaluate_reliever_segments(), and select_by_run().

Examples

```
# Fitting and evaluating the high-dimensional lasso path takes over five seconds.
set.seed(2026)
n <- 900
p <- 100
tau <- c(300, 600)
b0 <- c(3, -2.5, 2, -1.5, 1.5, rep(0, p - 5))
delta <- cbind(-2 * b0, 1.8 * b0)
data <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
holdout <- dgp_linear_regression(n, p, tau, b0, delta, sig = 1)$data
fit <- reliever(
  X = data[, -1, drop = FALSE], y = data[, 1],
  cpd_family = "lasso",
  cpn_max = 7, dm = 30, cov_rate = 0.6, # just for speed in example
  method = "SN"
)
selected <- select_holdout(
  result = fit,
  data = data[, -1, drop = FALSE], y = data[, 1],
  eval_data = holdout[, -1, drop = FALSE], eval_y = holdout[, 1]
)
selected
stopifnot(identical(selected$K_hat, 2L))
selected_at_k <- select_holdout(
  result = fit, data = data,
  eval_data = holdout, K = 2
)
selected_at_k
stopifnot(all(selected_at_k$K_hat == 2L))
```

---

summary.cv_reliever_result

Summarize a cross-validated Reliever result

Description

Return the final full-data segmentation chosen by outer cross-validation. This is the same compact one-row data frame stored in object$summary.

Usage

```
## S3 method for class 'cv_reliever_result'
summary(object, ...)
```

Arguments

object	A result returned by cv.reliever() or a related cross-validation wrapper.
...	Unused.

Value

A one-row data frame. The compact view contains `rule = "outer_cv"`, `K_hat`, the changepoint locations in list-column `cpd_hat`, the selected hyperparameter, WBS stopping threshold, or PELT/OP penalty when relevant, and `cv_mean` with `cv_se`. It includes `row_type` when statistically distinct output types were compared. The complete comparison is in `object$cv_loss`; its optional `run_id` maps to `object$full_data_fit$run_meta`. A selected `hyper_value` is printed under its model-specific `hyper_name`, such as `lambda`, when that metadata is available.

See Also

`cv.reliever()`, `cv.reliever_generic()`, `plot.cv_reliever_result()`

---

summary.reliever_result

Summarize a Reliever result

Description

Return the explicit selection stored in `object$summary`.

- With the ordinary `cpn_crit = "none"` default, K-indexed paths have an empty summary.
- An information-criterion fit reports its rule, estimated changepoints, and model setting. It selects K within each run, without comparing hyperparameters across runs.
- Supplied WBS thresholds or PELT/OP penalties retain their associated segmentations even when no K-selection criterion is requested.

Use `select_across_runs()` for joint selection across comparable paths.

Usage

```
## S3 method for class 'reliever_result'
summary(object, ...)
```

Arguments

`object`	A result returned by `reliever()` or a related wrapper.
`...`	Unused.

Value

The same compact data frame as `object$summary`, with one row per explicitly selected model setting or supplied search value. `K_hat` is the estimated number of changepoints and `cpd_hat` is a list-column containing their locations. A solution-path fit can return several rows, for example one per lambda, when its criterion selects K within each model setting rather than comparing settings. Printing that summary reports this distinction without adding another data-frame column. If the fitted object contains several statistically distinct loss-row types, `row_type` identifies the selected source, such as `"recv"`. A stored `hyper_value` is printed under the model-specific `hyper_name`, such as `lambda`, when that metadata is available.

See Also

`reliever()`, `plot.reliever_result()`, and `select_by_run()`.

`twostep` *Two-step changepoint search*

Description

Alternative to `reliever()` for WBS-family searches. Within each search interval, fit left and right models at only a small set of initial split fractions. Their observation-level losses are then reused to score every admissible split location. This can reduce model fitting when a custom model is expensive, at the cost of using the two-step approximation rather than exact interval fits.

Usage

```
twostep(
  data,
  reg_fun,
  cpn_max = 3,
  dm = 50,
  num_init = 3,
  init_cand = NULL,
  method = "WBS",
  M = 100,
  wbs_seed = NULL,
  wbs_stop_crit = NULL,
  echo = FALSE,
  detail = FALSE,
  ...
)
```

Arguments

`data`	Data object accepted by `reg_fun`.
`reg_fun`	Custom interval-loss function following the generic fitting contract. It must return observation-level losses; a fast block-loss-only C++ implementation is not sufficient.
`cpn_max`	Maximum number of changepoints.
`dm`	Minimum segment length.
`num_init`	Number of equally spaced initial split fractions used when `init_cand = NULL`.
`init_cand`	Optional numeric vector of initial split fractions in `(0, 1)`. Within each search interval, a fraction is mapped to the nearest split that leaves at least `dm` observations on both sides; duplicate mapped splits are fitted once.
`method`	Search algorithm. Supported methods are `"WBS"`, `"WBS_recursive"`, `"SeedBS"`, and `"BS"`. `"WBS"` uses Reliever's global greedy WBS path: after each split, it compares all current child segments and splits the segment with the largest gain. `"WBS_recursive"` follows the original recursive WBS style used by the WBS package.
`M`	Maximum retained random intervals for `"WBS"` and `"WBS_recursive"`. Ignored by `"SeedBS"` and `"BS"`.
`wbs_seed`	Optional seed for random WBS interval generation.

wbs_stop_crit	Optional stopping thresholds. For each threshold, keep the splits before the first proposed split whose gain is no larger than the threshold. The search remains limited to cpn_max splits.
echo	Print timing messages.
detail	When TRUE, retain gain_mat, the best gain found for each loss output and search interval, and split_mat, the corresponding split locations.
...	Additional arguments passed to reg_fun.

Value

A reliever_result object with the common components documented in reliever(). With wbs_stop_crit, summary() reports the segmentation for each threshold and loss output. Otherwise the summary is empty because this approximation has no whole-segmentation loss for selecting K; the candidates remain in cpd_path$candidates. Use select_holdout() when independent evaluation observations are available.

References

Kaul, A., Jandhyala, V. K., and Fotopoulos, S. B. (2019). An efficient two step algorithm for high dimensional change point regression models without grid search. *Journal of Machine Learning Research*, 20(111), 1–40.

See Also

reliever_generic(), select_holdout(), and local_refine().

Examples

```
set.seed(2026)
n_seg <- 300
x <- rbind(
  matrix(rnorm(n_seg * 5, mean = 0, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = 4, sd = 0.5), n_seg, 5),
  matrix(rnorm(n_seg * 5, mean = -4, sd = 0.5), n_seg, 5)
)
fit <- twostep(x, cpn_max = 7, dm = 30, reg_fun = reg_fun_mean,
               method = "SeedBS", echo = FALSE)
fit$cpd_path$candidates
```

Index